Q:- 1. Database Initialization and Table Creation: Create a New Database, and create tables: students table, courses table & enrollments table.

- **Input:-**
- **init_tables.py:-**

```
import sqlite3

def init_database():
    # 1. Connect to SQLite database (creates database file if it does not exist)
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # Enable Foreign Key Support
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 2. Create 'students' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        student_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER NOT NULL,
        major TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL
    );
    ''')

    # 3. Create 'courses' table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS courses (
        course_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_code TEXT UNIQUE NOT NULL,
        course_name TEXT NOT NULL,
        credits INTEGER NOT NULL
    );
    ''')

    # 4. Create 'enrollments' table with Foreign Keys
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS enrollments (
        enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id INTEGER NOT NULL,
        course_id INTEGER NOT NULL,
        grade TEXT,
```

```
        FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
RESTRICT,
        FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE
    );
    "")

    conn.commit()
    print("Database 'university.db' initialized successfully.")
    print("Tables created: students, courses, enrollments.")
    conn.close()

if __name__ == '__main__':
    init_database()
```

- **Output:-**

```
Database 'university.db' initialized successfully.
Tables created: students, courses, enrollments.
```

Q:- 2. Data Insertion (Create Operations): Insert Sample Data into students, courses, and enrollments tables.

- **Input:-**
- **insert_sample_data.py:-**

```
import sqlite3

def insert_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Insert sample students
    students_data = [
        ('Amit Sharma', 20, 'Computer Science', 'amit.sharma@example.com'),
        ('Priya Patel', 21, 'Information Technology', 'priya.patel@example.com'),
        ('Rahul Verma', 22, 'Electronics', 'rahul.verma@example.com'),
        ('Sneha Joshi', 20, 'Computer Science', 'sneha.joshi@example.com')
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO students (name, age, major, email)
    VALUES (?, ?, ?, ?);
    ''', students_data)

    # Insert sample courses
    courses_data = [
        ('CS501', 'Database Systems', 4),
        ('CS504', 'Web Development Using MEAN Stack', 4),
        ('IT502', 'Cloud Computing', 3)
    ]
    cursor.executemany('''
    INSERT OR IGNORE INTO courses (course_code, course_name, credits)
    VALUES (?, ?, ?);
    ''', courses_data)

    # Insert sample enrollments
    enrollments_data = [
        (1, 1, 'A'),
        (1, 2, 'B'),
        (2, 2, 'A'),
        (3, 3, 'B'),
        (4, 1, 'A')
    ]
```

```
    cursor.executemany('''
    INSERT INTO enrollments (student_id, course_id, grade)
    VALUES (?, ?, ?);
    ''', enrollments_data)

    conn.commit()
    print("Sample data inserted successfully into students, courses, and enrollments.")
    conn.close()

if __name__ == '__main__':
    insert_data()
```

- **Output:-**

Sample data inserted successfully into students, courses, and enrollments.

Q:- 3. Data Retrieval (Read Operations): 1. Basic Retrieval, 2. Filtered Retrieval, 3. Complex Queries (using JOINs).

- **Input:-**
- **retrieve_data.py:-**

```
import sqlite3

def retrieve_data():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Basic Retrieval - All Students
    print("--- 1. Basic Retrieval (All Students) ---")
    cursor.execute("SELECT student_id, name, major, email FROM students;")
    for row in cursor.fetchall():
        print(f"ID: {row[0]} | Name: {row[1]} | Major: {row[2]} | Email: {row[3]}")

    # 2. Filtered Retrieval - Students with Computer Science Major
    print("\n--- 2. Filtered Retrieval (Major = 'Computer Science') ---")
    cursor.execute("SELECT name, email FROM students WHERE major = 'Computer
Science';")
    for row in cursor.fetchall():
        print(f"Student: {row[0]} | Email: {row[1]}")

    # 3. Complex Query - JOIN students, enrollments, and courses
    print("\n--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---")
    query = '''
    SELECT s.name, c.course_code, c.course_name, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id
    ORDER BY s.name;
    '''
    cursor.execute(query)
    for row in cursor.fetchall():
        print(f"Student: {row[0]:<14} | Course: {row[1]} ({row[2]}) | Grade: {row[3]}")

    conn.close()

if __name__ == '__main__':
    retrieve_data()
```

➢ Output:-

```
--- 1. Basic Retrieval (All Students) ---
ID: 1 | Name: Amit Sharma | Major: Computer Science | Email: amit.sharma@example.com
ID: 2 | Name: Priya Patel | Major: Information Technology | Email: priya.patel@example.com
ID: 3 | Name: Rahul Verma | Major: Electronics | Email: rahul.verma@example.com
ID: 4 | Name: Sneha Joshi | Major: Computer Science | Email: sneha.joshi@example.com

--- 2. Filtered Retrieval (Major = 'Computer Science') ---
Student: Amit Sharma | Email: amit.sharma@example.com
Student: Sneha Joshi | Email: sneha.joshi@example.com

--- 3. Complex Queries (JOIN across Students, Enrollments, and Courses) ---
Student: Amit Sharma   | Course: CS501 (Database Systems) | Grade: A
Student: Amit Sharma   | Course: CS504 (Web Development Using MEAN Stack) | Grade: B
Student: Priya Patel   | Course: CS504 (Web Development Using MEAN Stack) | Grade: A
Student: Rahul Verma   | Course: IT502 (Cloud Computing) | Grade: B
Student: Sneha Joshi   | Course: CS501 (Database Systems) | Grade: A
```

Q:- 4. Data Update (Update Operations): 1. Update Student Major, 2. Update Enrollment Grade.

- **Input:-**
- **update_data.py:-**

```
import sqlite3

def update_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Update Student Major
    cursor.execute('''
    UPDATE students
    SET major = 'Data Science'
    WHERE name = 'Amit Sharma';
    ''')
    print("1. Student Major Updated: Amit Sharma changed to Data Science.")

    # 2. Update Enrollment Grade
    cursor.execute('''
    UPDATE enrollments
    SET grade = 'A+'
    WHERE student_id = 1 AND course_id = 2;
    ''')
    print("2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.")

    conn.commit()

    # Verification Query
    print("\nVerification of Updated Records:")
    cursor.execute("SELECT name, major FROM students WHERE name = 'Amit Sharma';")
    print("Updated Student:", cursor.fetchone())

    cursor.execute("SELECT student_id, course_id, grade FROM enrollments WHERE
student_id = 1 AND course_id = 2;")
    print("Updated Enrollment:", cursor.fetchone())

    conn.close()

if __name__ == '__main__':
    update_operations()
```

➢ Output:-

1. Student Major Updated: Amit Sharma changed to Data Science.
2. Enrollment Grade Updated: Student ID 1 for Course ID 2 updated to A+.

Verification of Updated Records:
Updated Student: ('Amit Sharma', 'Data Science')
Updated Enrollment: (1, 2)

Q:- 5. Data Deletion (Delete Operations): 1. Delete an Enrollment, 2. Attempt to Delete Enrolled Student (Observe FK Constraint), 3. Safe Student Deletion.

- **Input:-**
- **delete_operations.py:-**

```
import sqlite3

def delete_operations():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()
    cursor.execute("PRAGMA foreign_keys = ON;")

    # 1. Delete an Enrollment
    cursor.execute("DELETE FROM enrollments WHERE enrollment_id = 1;")
    conn.commit()
    print("1. Enrollment with ID 1 deleted successfully.")

    # 2. Attempt to Delete Enrolled Student (Observe FK Constraint)
    print("\n2. Attempting to delete student ID 2 who has active enrollments...")
    try:
        cursor.execute("DELETE FROM students WHERE student_id = 2;")
        conn.commit()
    except sqlite3.IntegrityError as e:
        print(f"FOREIGN KEY CONSTRAINT CAUGHT: {e}")
        print("Observation: The database rejected deletion because RESTRICT constraint
protects enrolled students.")

    # 3. Safe Student Deletion: Delete student's enrollments first, then student
    print("\n3. Executing Safe Student Deletion for Student ID 3:")
    cursor.execute("DELETE FROM enrollments WHERE student_id = 3;")
    cursor.execute("DELETE FROM students WHERE student_id = 3;")
    conn.commit()
    print("Enrollments for student 3 removed, followed by safe student deletion.")

    conn.close()

if __name__ == '__main__':
    delete_operations()
```

- **Output:-**

```
1. Enrollment with ID 1 deleted successfully.
```

2. Attempting to delete student ID 2 who has active enrollments...
FOREIGN KEY CONSTRAINT CAUGHT: FOREIGN KEY constraint failed
Observation: The database rejected deletion because RESTRICT constraint protects enrolled students.

3. Executing Safe Student Deletion for Student ID 3:
Enrollments for student 3 removed, followed by safe student deletion.

Q:- 6. Database Dumping: 1. Full Database Dump, 2. Schema-Only Dump, 3. Data-Only Dump.

- **Input:-**
- **database_dumps.py:-**

```
import sqlite3

def dump_database():
    conn = sqlite3.connect('university.db')

    # 1. Full Database Dump (Schema + Data)
    with open('full_dump.sql', 'w') as f:
        for line in conn.iterdump():
            f.write(f"{line}\n")
    print("1. Full Database Dump exported to 'full_dump.sql'.")

    # 2. Schema-Only Dump
    cursor = conn.cursor()
    cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name NOT
LIKE 'sqlite_%';")
    schema_statements = cursor.fetchall()
    with open('schema_dump.sql', 'w') as f:
        for stmt in schema_statements:
            if stmt[0]:
                f.write(f"{stmt[0]};\n\n")
    print("2. Schema-Only Dump exported to 'schema_dump.sql'.")

    # 3. Data-Only Dump (INSERT statements)
    with open('data_dump.sql', 'w') as f:
        for line in conn.iterdump():
            if line.startswith('INSERT INTO'):
                f.write(f"{line}\n")
    print("3. Data-Only Dump exported to 'data_dump.sql'.")

    conn.close()

if __name__ == '__main__':
    dump_database()
```

- **Output:-**

```
1. Full Database Dump exported to 'full_dump.sql'.
2. Schema-Only Dump exported to 'schema_dump.sql'.
3. Data-Only Dump exported to 'data_dump.sql'.
```

Q:- 7. CSV File Handling: 1. Export Enrollments to CSV, 2. Import New Students from CSV.

- **Input:-**
- **csv_handling.py:-**

```
import sqlite3
import csv

def handle_csv():
    conn = sqlite3.connect('university.db')
    cursor = conn.cursor()

    # 1. Export Enrollments to CSV
    cursor.execute('''
    SELECT e.enrollment_id, s.name, c.course_code, e.grade
    FROM enrollments e
    JOIN students s ON e.student_id = s.student_id
    JOIN courses c ON e.course_id = c.course_id;
    ''')
    rows = cursor.fetchall()
    with open('enrollments_export.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['Enrollment_ID', 'Student_Name', 'Course_Code', 'Grade'])
        writer.writerows(rows)
    print("1. Enrollments exported successfully to 'enrollments_export.csv'.")

    # 2. Import New Students from CSV
    # Create sample import CSV
    new_students = [
        ['Vikram Malhotra', 21, 'Cyber Security', 'vikram.m@example.com'],
        ['Ananya Sen', 20, 'Data Science', 'ananya.s@example.com']
    ]
    with open('new_students.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['name', 'age', 'major', 'email'])
        writer.writerows(new_students)

    with open('new_students.csv', 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            cursor.execute('''
            INSERT OR IGNORE INTO students (name, age, major, email)
            VALUES (?, ?, ?, ?);
```

```
            ''', (row['name'], row['age'], row['major'], row['email']))

    conn.commit()
    print("2. New students imported successfully from 'new_students.csv'.")

    # Verification
    cursor.execute("SELECT student_id, name, major FROM students WHERE major IN
('Cyber Security', 'Data Science');")
    print("\nNewly Imported Students Verified:")
    for r in cursor.fetchall():
        print(f"ID: {r[0]} | Name: {r[1]} | Major: {r[2]}")

    conn.close()

if __name__ == '__main__':
    handle_csv()
```

- **Output:-**

```
1. Enrollments exported successfully to 'enrollments_export.csv'.
2. New students imported successfully from 'new_students.csv'.

Newly Imported Students Verified:
ID: 1 | Name: Amit Sharma | Major: Data Science
ID: 5 | Name: Vikram Malhotra | Major: Cyber Security
ID: 6 | Name: Ananya Sen | Major: Data Science
```

Journal Assignment – 3

Q:- 1. Setup Your Database Connection: 1. Import Module, 2. Connect to Database, 3. Create a Cursor.

- **Input:-**
- **step1_setup_connection.py:-**

```
# 1. Import Module
import sqlite3

# 2. Connect to Database
connection = sqlite3.connect('store.db')
print("Successfully connected to 'store.db'.")

# 3. Create a Cursor
cursor = connection.cursor()
print("Cursor created successfully to execute SQL statements.")

# Close connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Successfully connected to 'store.db'.
Cursor created successfully to execute SQL statements.
Connection closed
```

Q:- 2. Create Your First Table (products): 1. Define SQL, 2. Execute SQL, 3. Save Changes, 4. Close Connection.

- **Input:-**
- **step2_create_table.py:-**

```
import sqlite3

connection = sqlite3.connect('store.db')
cursor = connection.cursor()

# 1. Define SQL
create_table_sql = '''
CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    quantity INTEGER NOT NULL
);
'''

# 2. Execute SQL
cursor.execute(create_table_sql)
print("Table 'products' created successfully.")

# 3. Save Changes
connection.commit()
print("Changes saved (committed).")

# 4. Close Connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Table 'products' created successfully.
Changes saved (committed).
Connection closed.
```

Q:- 3. Add New Products (Create Operation - INSERT): 1. Re-connect and Get Cursor, 2. Insert Single Product, 3. Insert Multiple Products, 4. Confirm Insertions, 5. Commit and Close.

- **Input:-**
- **step3_insert_products.py:-**

```
import sqlite3
# 1. Re-connect and Get Cursor
conn = sqlite3.connect('store.db')
cur = conn.cursor(
# 2. Insert Single Product
cur.execute('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', ('Wireless Mouse', 450.00, 25))
print("Single product inserted: Wireless Mouse.")
 # 3. Insert Multiple Products
products_list = [
    ('Mechanical Keyboard', 2200.00, 15),
    ('HD Webcam', 1800.00, 5),
    ('USB-C Hub', 1250.00, 4),
    ('Gaming Headset', 2800.00, 18)
]
cur.executemany('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', products_list)
print("Multiple products inserted successfully.")

# 4. Confirm Insertions
cur.execute("SELECT COUNT(*) FROM products;")
total_count = cur.fetchone()[0]
print(f"Total products now in table: {total_count}")
# 5. Commit and Close
conn.commit()
conn.close()
print("Transaction committed and connection closed.")
```

- **Output:-**

```
Single product inserted: Wireless Mouse.
Multiple products inserted successfully.
Total products now in table: 5
Transaction committed and connection closed.
```

Q:- 4. View Your Products (Read Operation - SELECT): 1. Print Products, 2. Select Specific Product, 3. Select Products with Low Quantity.

- **Input:-**
- **step4_view_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Print All Products
print("--- 1. All Products in Inventory ---")
cur.execute("SELECT * FROM products;")
for row in cur.fetchall():
    print(f"ID: {row[0]:2d} | Name: {row[1]:<20} | Price: Rs.{row[2]:7.2f} | Quantity: {row[3]}"
# 2. Select Specific Product by Name
print("\n--- 2. Select Specific Product ('Mechanical Keyboard') ---")
cur.execute("SELECT * FROM products WHERE product_name = ?;", ('Mechanical
Keyboard',))
prod = cur.fetchone()
print(f"Found: ID: {prod[0]} | Name: {prod[1]} | Price: Rs.{prod[2]} | Stock: {prod[3]}")

# 3. Select Products with Low Quantity (Quantity < 10)
print("\n--- 3. Products with Low Stock (Quantity < 10) ---")
cur.execute("SELECT product_name, quantity FROM products WHERE quantity < 10;")
for p in cur.fetchall():
    print(f"Low Stock Alert: {p[0]} (Only {p[1]} units left!)")

conn.close()
```

- **Output:-**

```
--- 1. All Products in Inventory ---
ID:  1 | Name: Wireless Mouse       | Price: Rs. 450.00 | Quantity: 25
ID:  2 | Name: Mechanical Keyboard  | Price: Rs.2200.00 | Quantity: 15
ID:  3 | Name: HD Webcam            | Price: Rs.1800.00 | Quantity: 5
ID:  4 | Name: USB-C Hub            | Price: Rs.1250.00 | Quantity: 4
ID:  5 | Name: Gaming Headset       | Price: Rs.2800.00 | Quantity: 18
--- 2. Select Specific Product ('Mechanical Keyboard') ---
Found: ID: 2 | Name: Mechanical Keyboard | Price: Rs.2200.0 | Stock: 15

--- 3. Products with Low Stock (Quantity < 10) ---
Low Stock Alert: HD Webcam (Only 5 units left!)
Low Stock Alert: USB-C Hub (Only 4 units left!)
```

Q:- 5. Update Product Information (Update Operation - UPDATE): 1. Update Quantity, 2. Update Price, 3. Confirm Updates.

- **Input:-**
- **step5_update_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Update Quantity of HD Webcam
cur.execute('''
UPDATE products
SET quantity = 30
WHERE product_name = 'HD Webcam';
''')
print("1. Quantity of 'HD Webcam' updated to 30.")

# 2. Update Price of Wireless Mouse
cur.execute('''
UPDATE products
SET price = 499.00
WHERE product_name = 'Wireless Mouse';
''')
print("2. Price of 'Wireless Mouse' updated to Rs. 499.00.")

conn.commit()

# 3. Confirm Updates
print("\n--- 3. Confirmation of Updated Products ---")
cur.execute("SELECT product_name, price, quantity FROM products WHERE product_name
IN ('HD Webcam', 'Wireless Mouse');")
for r in cur.fetchall():
    print(f"Product: {r[0]:<15} | New Price: Rs.{r[1]} | New Quantity: {r[2]}")

conn.close()
```

- **Output:-**

```
1. Quantity of 'HD Webcam' updated to 30.
2. Price of 'Wireless Mouse' updated to Rs. 499.00.
--- 3. Confirmation of Updated Products ---
Product: Wireless Mouse  | New Price: Rs.499.0 | New Quantity: 25
Product: HD Webcam       | New Price: Rs.1800.0 | New Quantity: 30
```

Q:- 6. Remove Products (Delete Operation - DELETE): 1. Delete a Product, 2. Confirm Deletion.

- **Input:-**
- **step6_delete_product.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Delete a Product (USB-C Hub)
cur.execute("DELETE FROM products WHERE product_name = 'USB-C Hub';")
conn.commit()
print("1. Product 'USB-C Hub' deleted from database.")

# 2. Confirm Deletion
print("\n--- 2. Inventory Check after Deletion ---")
cur.execute("SELECT product_id, product_name FROM products;")
remaining = cur.fetchall()
for p in remaining:
    print(f"ID: {p[0]} | Name: {p[1]}")

conn.close()
```

- **Output:-**

```
1. Product 'USB-C Hub' deleted from database.

--- 2. Inventory Check after Deletion ---
ID: 1 | Name: Wireless Mouse
ID: 2 | Name: Mechanical Keyboard
ID: 3 | Name: HD Webcam
ID: 5 | Name: Gaming Headset
```

Q:- 7. Understanding commit() and rollback(): 1. Demonstrate Commit, 2. Demonstrate Rollback.

- **Input:-**
- **step7_commit_and_rollback.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Demonstrate Commit
print("--- 1. Demonstrating commit() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Bluetooth
Speaker', 1499.0, 10);")
conn.commit()  # Changes are permanently saved
print("Product 'Bluetooth Speaker' inserted and committed.")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Bluetooth
Speaker';")
print("Found after commit:", cur.fetchone()[0])

# 2. Demonstrate Rollback
print("\n--- 2. Demonstrating rollback() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Accidental
Dummy Product', 99999.0, 0);")
print("Inserted 'Accidental Dummy Product' (uncommitted).")

# Rollback changes before commit
conn.rollback()
print("Transaction rolled back (changes reverted)!")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Accidental
Dummy Product';")
result = cur.fetchone()
print("Searching for uncommitted product after rollback:", result)
print("Observation: The uncommitted record was completely removed by rollback().")

conn.close()
```

- **Output:-**

```
--- 1. Demonstrating commit() ---
Product 'Bluetooth Speaker' inserted and committed.
Found after commit: Bluetooth Speaker
```

--- 2. Demonstrating rollback() ---
Inserted 'Accidental Dummy Product' (uncommitted).
Transaction rolled back (changes reverted)!
Searching for uncommitted product after rollback: None
Observation: The uncommitted record was completely removed by rollback().

Q:- 8. Basic Database Backup (Dump): 1. Dump Database, 2. Save to File.

- **Input:-**
- **step8_dump_backup.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')

# 1. Dump Database & 2. Save to File
backup_filename = 'store_backup.sql'
with open(backup_filename, 'w') as f:
    for line in conn.iterdump():
        f.write(f"{line}\n")

print(f"Database successfully dumped and backed up to '{backup_filename}'.")

# Preview the backup file content
print("\n--- Backup File Preview (First 5 lines) ---")
with open(backup_filename, 'r') as f:
    for i in range(5):
        print(f.readline().strip())

conn.close()
```

- **Output:-**

```
Database successfully dumped and backed up to 'store_backup.sql'.

--- Backup File Preview (First 5 lines) ---
BEGIN TRANSACTION;
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
```

Q:- 9. Export Products to CSV File.

- **Input:-**
- **step9_export_csv.py:-**

```
import sqlite3
```

```
import csv

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# Retrieve all products
cur.execute("SELECT * FROM products;")
rows = cur.fetchall()

csv_file = 'products_exported.csv'
with open(csv_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Product_ID', 'Product_Name', 'Price', 'Quantity'])
    writer.writerows(rows)

print(f"Products exported successfully to '{csv_file}'.")

# Print CSV content
print("\n--- Exported CSV Contents ---")
with open(csv_file, 'r') as f:
    print(f.read())

conn.close()
```

- **Output:-**

```
Products exported successfully to 'products_exported.csv'.

--- Exported CSV Contents ---
Product_ID,Product_Name,Price,Quantity
1,Wireless Mouse,499.0,25
2,Mechanical Keyboard,2200.0,15
3,HD Webcam,1800.0,30
5,Gaming Headset,2800.0,18
6,Bluetooth Speaker,1499.0,10
```

Journal Assignment – 4

Q:- 1. Setup - Database Connection and Table Creation: 1. Connect to Database, 2. Create tasks Table.

- **Input:-**
- **step1_setup_todo_db.py:-**

```
import sqlite3

def setup_todo_database():
    # 1. Connect to Database (creates 'todo_app.db' if it does not exist)
    conn = sqlite3.connect('todo_app.db')
    cursor = conn.cursor()
    print("Connected to database 'todo_app.db' successfully.")

    # 2. Create 'tasks' Table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS tasks (
        task_id INTEGER PRIMARY KEY AUTOINCREMENT,
        description TEXT NOT NULL,
        due_date TEXT NOT NULL,
        is_completed INTEGER DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
    ''')
    conn.commit()
    print("Table 'tasks' created successfully with fields: task_id, description, due_date,
is_completed, created_at.")
    conn.close()
    print("Connection closed.")

if __name__ == '__main__':
    setup_todo_database()
```

- **Output:-**

```
Connected to database 'todo_app.db' successfully.
Table 'tasks' created successfully with fields: task_id, description, due_date, is_completed,
created_at.
Connection closed.
```

Q:- 2. Core To-Do List Functions: 1. add_task(description, due_date), 2. view_tasks(show_completed=False), 3. mark_task_completed(task_id), 4. delete_task(task_id).

- **Input:-**
- **step2_todo_functions.py:-**

```
import sqlite3

DB_NAME = 'todo_app.db'

def get_connection():
    return sqlite3.connect(DB_NAME)

# 1. add_task(description, due_date) function
def add_task(description, due_date):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('''
    INSERT INTO tasks (description, due_date, is_completed)
    VALUES (?, ?, 0);
    ''', (description, due_date))
    conn.commit()
    task_id = cursor.lastrowid
    conn.close()
    print(f"Task #{task_id} ('{description}') added successfully.")
    return task_id

# 2. view_tasks(show_completed=False) function
def view_tasks(show_completed=False):
    conn = get_connection()
    cursor = conn.cursor()
    if show_completed:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
ORDER BY task_id;")
    else:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
WHERE is_completed = 0 ORDER BY task_id;")

    rows = cursor.fetchall()
    conn.close()

    print("-" * 65)
    print(f"{'ID':<5} {'Description':<32} {'Due Date':<14} {'Status'}")
```

```
    print("-" * 65)
    if not rows:
        print("No tasks found.")
    else:
        for row in rows:
            status = "Completed" if row[3] == 1 else "Pending"
            print(f"{row[0]:<5} {row[1]:<32} {row[2]:<14} {status}")
    print("-" * 65)

# 3. mark_task_completed(task_id) function
def mark_task_completed(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE tasks SET is_completed = 1 WHERE task_id = ?;", (task_id,))
    conn.commit()
    rows_affected = cursor.rowcount
    conn.close()
    if rows_affected > 0:
        print(f"Task #{task_id} marked as completed.")
    else:
        print(f"Task #{task_id} not found.")

# 4. delete_task(task_id) function
def delete_task(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM tasks WHERE task_id = ?;", (task_id,))
    conn.commit()
    rows_affected = cursor.rowcount
    conn.close()
    if rows_affected > 0:
        print(f"Task #{task_id} deleted successfully.")
    else:
        print(f"Task #{task_id} not found.")

# Demonstration
if __name__ == '__main__':
    t1 = add_task("Complete ML Practical Assignment", "2026-10-25")
    t2 = add_task("Prepare Database Presentation", "2026-10-28")
    t3 = add_task("Submit Web Tech Project", "2026-10-30")

    print("\nPending Tasks:")
    view_tasks(show_completed=False)
```

```
print("\nMarking Task #1 as completed:")
mark_task_completed(t1)
print("\nAll Tasks (including completed):")
view_tasks(show_completed=True)
print("\nDeleting Task #3:")
delete_task(t3)

print("\nRemaining Tasks:")
view_tasks(show_completed=True)
```

- **Output:-**

```
Task #1 ('Complete ML Practical Assignment') added successfully.
Task #2 ('Prepare Database Presentation') added successfully.
Task #3 ('Submit Web Tech Project') added successfully.
 Pending Tasks:
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Pending
2   Prepare Database Presentation  2026-10-28   Pending
3   Submit Web Tech Project       2026-10-30   Pending
------------------------------------------------------------
 Marking Task #1 as completed:
Task #1 marked as completed.
 All Tasks (including completed):
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Completed
2   Prepare Database Presentation  2026-10-28   Pending
3   Submit Web Tech Project       2026-10-30   Pending
------------------------------------------------------------

Deleting Task #3:
Task #3 deleted successfully.

Remaining Tasks:
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Completed
2   Prepare Database Presentation  2026-10-28   Pending
------------------------------------------------------------
```

Q:- 3. Simple User Interface (Main Program Loop): 1. Display Menu (6 options), 2. Enter choice, 3. Process Choice, 4. Error Handling for Input.

- **Input:-**
- **step3_todo_cli_app.py:-**

```
import sqlite3
import sys

DB_NAME = 'todo_app.db'

def get_connection():
    return sqlite3.connect(DB_NAME)

def display_menu():
    print("\n" + "=" * 32)
    print("      To-Do List Manager")
    print("=" * 32)
    print("1. Add New Task")
    print("2. View Pending Tasks")
    print("3. View All Tasks (including completed)")
    print("4. Mark Task as Completed")
    print("5. Delete Task")
    print("6. Exit")
    print("=" * 32)

def add_task(description, due_date):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO tasks (description, due_date, is_completed) VALUES (?, ?,
0);", (description, due_date))
    conn.commit()
    task_id = cursor.lastrowid
    conn.close()
    print(f"Success: Task #{task_id} added.")

def view_tasks(show_completed=False):
    conn = get_connection()
    cursor = conn.cursor()
    if show_completed:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
ORDER BY task_id;")
    else:
```

```
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
WHERE is_completed = 0 ORDER BY task_id;")
    rows = cursor.fetchall()
    conn.close()

    print("\nID   Status     Due Date      Description")
    print("-" * 55)
    if not rows:
        print("No tasks found.")
    for r in rows:
        st = "[DONE]" if r[3] == 1 else "[TODO]"
        print(f"{r[0]:<4} {st:<10} {r[2]:<13} {r[1]}")
    print("-" * 55)

def mark_task_completed(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE tasks SET is_completed = 1 WHERE task_id = ?;", (task_id,))
    conn.commit()
    updated = cursor.rowcount
    conn.close()
    if updated > 0:
        print(f"Success: Task #{task_id} marked as completed.")
    else:
        print(f"Error: Task #{task_id} does not exist.")

def delete_task(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM tasks WHERE task_id = ?;", (task_id,))
    conn.commit()
    deleted = cursor.rowcount
    conn.close()
    if deleted > 0:
        print(f"Success: Task #{task_id} deleted successfully.")
    else:
        print(f"Error: Task #{task_id} does not exist.")

def main_loop(simulated_inputs=None):
    # Simulated inputs allow demonstrating menu flow deterministically
    input_index = 0
    def custom_input(prompt):
        nonlocal input_index
        if simulated_inputs is not None and input_index < len(simulated_inputs):
```

```
        val = simulated_inputs[input_index]
        print(f"{prompt}{val}")
        input_index += 1
        return val
    return input(prompt)

while True:
    display_menu()
    try:
        choice = custom_input("Enter your choice: ").strip()

        # Error Handling for Empty Input
        if not choice:
            print("Invalid input. Please enter a number between 1 and 6.")
            continue

        if choice == '1':
            desc = custom_input("Enter task description: ").strip()
            if not desc:
                print("Error: Task description cannot be empty.")
                continue
            due = custom_input("Enter due date (YYYY-MM-DD): ").strip()
            add_task(desc, due)

        elif choice == '2':
            view_tasks(show_completed=False)

        elif choice == '3':
            view_tasks(show_completed=True)

        elif choice == '4':
            task_id_str = custom_input("Enter Task ID to mark completed: ").strip()
            if not task_id_str.isdigit():
                print("Error: Task ID must be a numeric value.")
                continue
            mark_task_completed(int(task_id_str))

        elif choice == '5':
            task_id_str = custom_input("Enter Task ID to delete: ").strip()
            if not task_id_str.isdigit():
                print("Error: Task ID must be a numeric value.")
                continue
            delete_task(int(task_id_str))
```

```
            elif choice == '6':
                print("Exiting To-Do List Manager. Goodbye!")
                break

            else:
                print("Error: Invalid choice. Please enter a digit from 1 to 6.")

        except Exception as e:
            print(f"Unexpected error occurred: {e}")

if __name__ == '__main__':
    # Sample run simulating user interactions
    sample_inputs = [
        "1", "Read Chapter 4 Database Indexing", "2026-10-26",
        "2",
        "4", "1",
        "3",
        "6"
    ]
    main_loop(simulated_inputs=sample_inputs)
```

- **Output:-**

```
==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 1
Enter task description: Read Chapter 4 Database Indexing
Enter due date (YYYY-MM-DD): 2026-10-26
Success: Task #1 added.

==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
```

```
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 2

ID   Status     Due Date      Description
-------------------------------------------------------
1    [TODO]     2026-10-26    Read Chapter 4 Database Indexing
-------------------------------------------------------

==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 4
Enter Task ID to mark completed: 1
Success: Task #1 marked as completed.

==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 3

ID   Status     Due Date      Description
-------------------------------------------------------
1    [DONE]     2026-10-26    Read Chapter 4 Database Indexing
-------------------------------------------------------
```

```
==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 6
Exiting To-Do List Manager. Goodbye!
```

Q – 1 Tkinter Basic Registration Form

- **Input:-**

```
from tkinter import *

# Main window
root = Tk()
root.title("Tkinter Widgets")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)

# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
```

```
# Gender label
gender_lbl = Label(
    root,
    text="Select a Gender:",
    bg="lightblue"
)
gender_lbl.pack(pady=5)

# Variable for radio buttons
gender = StringVar()
gender.set("Male")

r1 = Radiobutton(
    root,
    text="Male",
    variable=gender,
    value="Male",
    bg="lightblue"
)
r1.pack()

r2 = Radiobutton(
    root,
    text="Female",
    variable=gender,
    value="Female",
    bg="lightblue"
)
r2.pack()

# Check button
agree = IntVar()

check = Checkbutton(
    root,
    text="I agree",
    variable=agree,
    font=("Arial", 11),
    bg="lightblue"
```

```
)
check.pack(pady=10)

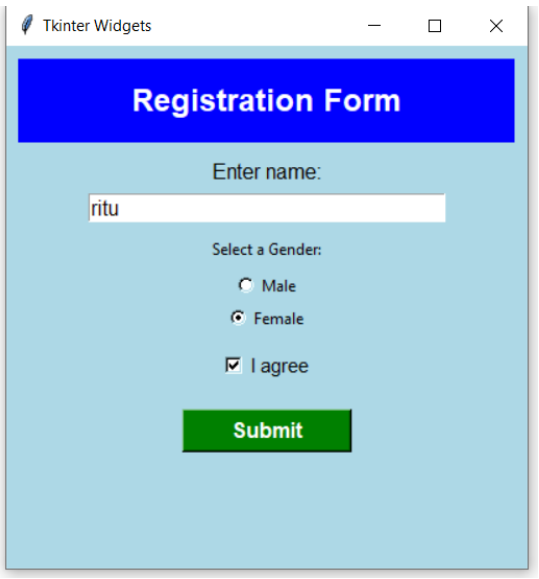
# Button function
def show_data():
    name = name_entry.get()
    print("Name:", name)
    print("Gender:", gender.get())
    print("Agree:", agree.get())

# Submit button
btn = Button(
    root,
    text="Submit",
    command=show_data,
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.pack(pady=10)

# Run the application
root.mainloop()
```

- **Output:-**

```
Name: palak
Gender: Female
Agree: 1
Name: ritu
Gender: Female
Agree: 1
```



Q – 2 Tkinter Registration Form with Country & Password

- **Input:-**

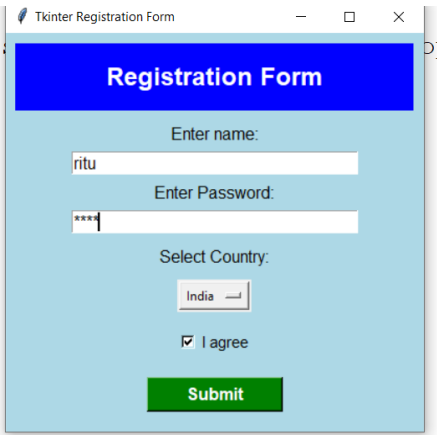
```
from tkinter import *
# Main window
root = Tk()
root.title("Tkinter Registration Form")
root.geometry("400x500")
root.configure(bg="lightblue")
# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)
# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()
# Name entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
# Password label
password_lbl = Label(
    root,
    text="Enter Password:",
```

```
    font=("Arial", 12),
    bg="lightblue"
)
password_lbl.pack()
# Password entry widget
password_entry = Entry(
    root,
    width=30,
    font=("Arial", 12),
    show="*"
)
password_entry.pack(pady=5)
# Country label
country_lbl = Label(
    root,
    text="Select Country:",
    font=("Arial", 12),
    bg="lightblue"
)
country_lbl.pack(pady=5)
# Variable for country
country = StringVar()
country.set("India")
# Country dropdown menu
country_menu = OptionMenu(
    root,
    country,
    "India",
    "USA",
    "UK",
    "Canada",
    "Australia"
)
country_menu.pack(pady=5)
# Check button
agree = IntVar()
check = Checkbutton(
    root,
    text="I agree",
    variable=agree,
```

```
    font=("Arial", 11),
    bg="lightblue"
)
check.pack(pady=10)
# Button function
def show_data():
    name = name_entry.get()
    password = password_entry.get()
    print("Name:", name)
    print("Password:", password)
    print("Country:", country.get())
    print("Agree:", agree.get())
# Submit button
btn = Button(
    root,
    text="Submit",
    command=show_data,
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.pack(pady=10)
# Run the application
root.mainloop()
```

- **Output:-**

```
== RESTART: C:/U                                    op
Name: palak
Password: 1234
Country: USA
Agree: 1
Name: ritu
Password: 1234
Country: India
Agree: 1
```



Q – 3 Tkinter Registration Form with Language Selection

- **Input:-**

```
from tkinter import *

# Main window
root = Tk()
root.title("Tkinter Registration Form")
root.geometry("400x500")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)

# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

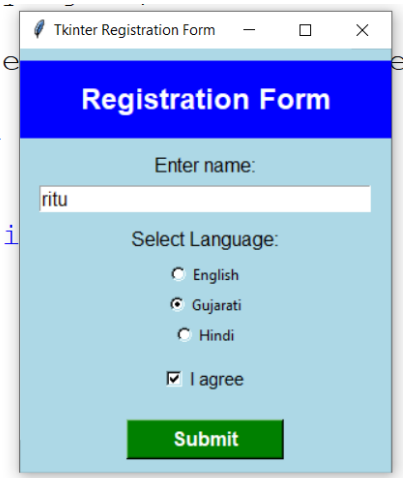
# Entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
```

```
# Language label
language_lbl = Label(
    root,
    text="Select Language:",
    font=("Arial", 12),
    bg="lightblue"
)
language_lbl.pack(pady=5)
# Variable for language
language = StringVar()
language.set("English")
# Radio buttons for language selection
r1 = Radiobutton(
    root,
    text="English",
    variable=language,
    value="English",
    bg="lightblue"
)
r1.pack()
r2 = Radiobutton(
    root,
    text="Gujarati",
    variable=language,
    value="Gujarati",
    bg="lightblue"
)
r2.pack()
r3 = Radiobutton(
    root,
    text="Hindi",
    variable=language,
    value="Hindi",
    bg="lightblue"
)
r3.pack()
# Check button
agree = IntVar()
check = Checkbutton(
```

```
    root,
    text="I agree",
    variable=agree,
    font=("Arial", 11),
    bg="lightblue"
)
check.pack(pady=10)
# Button function
def show_data():
    name = name_entry.get()
    print("Name:", name)
    print("Language:", language.get())
    print("Agree:", agree.get())
# Submit button
btn = Button(
    root,
    text="Submit",
    command=show_data,
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.pack(pady=10)
# Run the application
root.mainloop()
```

➢ Output:-

```
>>>
== RESTART: C:/Use
Name: palak
Language: English
Agree: 1
Name: ritu
Language: Gujarati
Agree: 1
```



Q – 4 Student Registration Form with SQLite (CRUD Operations)

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER,
    course TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Student Registration Form")
root.geometry("450x600")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Student Registration Form",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)
```

```
# Student ID label
id_lbl = Label(
    root,
    text="Enter Student ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Student ID entry
id_entry = Entry(root, width=30, font=("Arial", 12))
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(root, width=30, font=("Arial", 12))
name_entry.pack(pady=5)

# Age label
age_lbl = Label(
    root,
    text="Enter Age:",
    font=("Arial", 12),
    bg="lightblue"
)
age_lbl.pack()

# Age entry
age_entry = Entry(root, width=30, font=("Arial", 12))
age_entry.pack(pady=5)
```

```
# Course label
course_lbl = Label(
    root,
    text="Enter Course:",
    font=("Arial", 12),
    bg="lightblue"
)
course_lbl.pack()

# Course entry
course_entry = Entry(root, width=30, font=("Arial", 12))
course_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    age_entry.delete(0, END)
    course_entry.delete(0, END)


# Insert function
def insert_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not age or not course:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    try:
        age = int(age)
        if age <= 0:
            raise ValueError

        cursor.execute(
            "INSERT INTO students (name, age, course) VALUES (?, ?, ?)",
            (name, age, course)
```

```
            )
            conn.commit()

            messagebox.showinfo("Success", "Student inserted successfully")
            clear_fields()

        except ValueError:
            messagebox.showerror("Error", "Enter a valid age")


    # View function
    def view_data():
        view_win = Toplevel(root)
        view_win.title("Student Records")
        view_win.geometry("500x350")
        view_win.configure(bg="white")

        Label(
            view_win,
            text="Student Records (ID, Name, Age, Course)",
            font=("Arial", 12, "bold"),
            bg="white"
        ).pack(pady=10)

        cursor.execute("SELECT * FROM students")
        records = cursor.fetchall()

        if not records:
            Label(
                view_win,
                text="No student records found",
                bg="white"
            ).pack()
            return

        for record in records:
            Label(
                view_win,
                text=f"ID: {record[0]} | Name: {record[1]} | "
                     f"Age: {record[2]} | Course: {record[3]}",
```

```
            font=("Arial", 10),
            bg="white",
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    student_id = id_entry.get().strip()
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    course = course_entry.get().strip()

    if not student_id or not name or not age or not course:
        messagebox.showwarning(
            "Warning",
            "Enter Student ID, Name, Age and Course"
        )
        return

    try:
        student_id = int(student_id)
        age = int(age)

        if student_id <= 0 or age <= 0:
            raise ValueError

        cursor.execute(
            "UPDATE students SET name=?, age=?, course=? WHERE id=?",
            (name, age, course, student_id)
        )
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success",
                "Student updated successfully"
            )
            clear_fields()
        else:
```

```
            messagebox.showwarning(
                "Warning",
                "Student ID not found"
            )

        except ValueError:
            messagebox.showerror(
                "Error",
                "Student ID and Age must be valid positive numbers"
            )


    # Delete function
    def delete_data():
        student_id = id_entry.get().strip()

        if not student_id:
            messagebox.showwarning(
                "Warning",
                "Enter Student ID to delete"
            )
            return

        try:
            student_id = int(student_id)

            if student_id <= 0:
                raise ValueError

            cursor.execute(
                "SELECT * FROM students WHERE id=?",
                (student_id,)
            )
            record = cursor.fetchone()

            if record is None:
                messagebox.showwarning(
                    "Warning",
                    "Student ID not found"
                )
```

```
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete student {record[1]} (ID: {student_id})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM students WHERE id=?",
                (student_id,)
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Student deleted successfully"
            )
            clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Student ID"
        )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=5)

# View button
Button(
```

```
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=5)

# Update button
Button(
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=5)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=5)

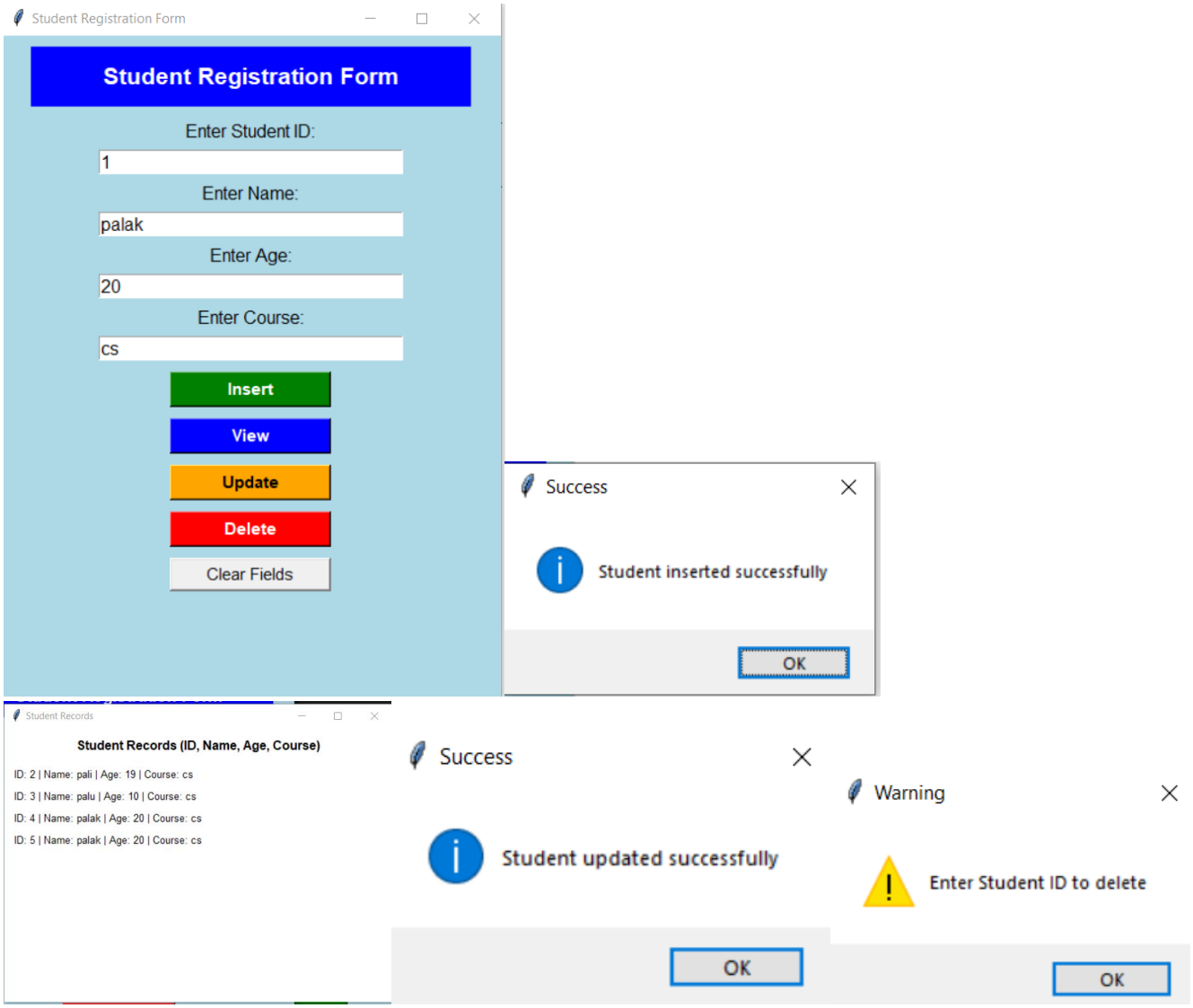
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=5)
```

```
# Close database connection
def close_app():
    conn.close()
    root.destroy()


root.protocol("WM_DELETE_WINDOW", close_app)

# Run application
root.mainloop()
```

➢ **Output:-**



Q – 5 Student Registration with SQLite (College Database)

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("college.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    roll_no INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    semester INTEGER
)
""")
conn.commit()

# Main window
root = Tk()
root.title("College Student Registration")
root.geometry("450x600")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="College Student Registration",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)

# Roll number label
```

```
roll_lbl = Label(
    root,
    text="Enter Roll Number:",
    font=("Arial", 12),
    bg="lightblue"
)
roll_lbl.pack()

# Roll number entry
roll_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
roll_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Student Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)

# Department label
dept_lbl = Label(
    root,
    text="Enter Department:",
    font=("Arial", 12),
    bg="lightblue"
)
```

```
dept_lbl.pack()

# Department entry
dept_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
dept_entry.pack(pady=5)

# Semester label
sem_lbl = Label(
    root,
    text="Enter Semester:",
    font=("Arial", 12),
    bg="lightblue"
)
sem_lbl.pack()

# Semester entry
sem_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
sem_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    roll_entry.delete(0, END)
    name_entry.delete(0, END)
    dept_entry.delete(0, END)
    sem_entry.delete(0, END)


# Insert function
def insert_data():
    roll = roll_entry.get().strip()
    name = name_entry.get().strip()
```

```
    dept = dept_entry.get().strip()
    sem = sem_entry.get().strip()

    if not roll or not name or not dept or not sem:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    try:
        roll = int(roll)
        sem = int(sem)

        if roll <= 0 or sem < 1 or sem > 8:
            raise ValueError

        cursor.execute(
            "INSERT INTO students VALUES (?, ?, ?, ?)",
            (roll, name, dept, sem)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Student registered successfully"
        )
        clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Roll Number and Semester (1-8)"
        )
    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Roll Number already exists"
        )


# View function
def view_data():
```

```
    view_win = Toplevel(root)
    view_win.title("College Student Records")
    view_win.geometry("550x350")

    Label(
        view_win,
        text="College Student Records",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute("SELECT * FROM students ORDER BY roll_no")
    records = cursor.fetchall()

    if not records:
        Label(view_win, text="No student records found").pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"Roll No: {record[0]} | Name: {record[1]} | "
                 f"Department: {record[2]} | Semester: {record[3]}",
            font=("Arial", 10),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    roll = roll_entry.get().strip()
    name = name_entry.get().strip()
    dept = dept_entry.get().strip()
    sem = sem_entry.get().strip()

    if not roll or not name or not dept or not sem:
        messagebox.showwarning(
            "Warning",
            "Enter Roll Number and all new details"
        )
        return
```

```
    try:
        roll = int(roll)
        sem = int(sem)

        if roll <= 0 or sem < 1 or sem > 8:
            raise ValueError

        cursor.execute(
            """UPDATE students
               SET name=?, department=?, semester=?
               WHERE roll_no=?""",
            (name, dept, sem, roll)
        )
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success",
                "Student record updated successfully"
            )
            clear_fields()
        else:
            messagebox.showwarning(
                "Warning",
                "Roll Number not found"
            )

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter valid Roll Number and Semester (1-8)"
        )


# Delete function
def delete_data():
    roll = roll_entry.get().strip()

    if not roll:
```

```
        messagebox.showwarning(
            "Warning",
            "Enter Roll Number to delete"
        )
        return

    try:
        roll = int(roll)

        if roll <= 0:
            raise ValueError

        cursor.execute(
            "SELECT name FROM students WHERE roll_no=?",
            (roll,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
                "Roll Number not found"
            )
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete student {record[0]} (Roll No: {roll})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM students WHERE roll_no=?",
                (roll,)
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Student record deleted successfully"
```

```
            )
            clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Roll Number"
        )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

# View button
Button(
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)

# Update button
Button(
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
```

```
).pack(pady=4)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=4)

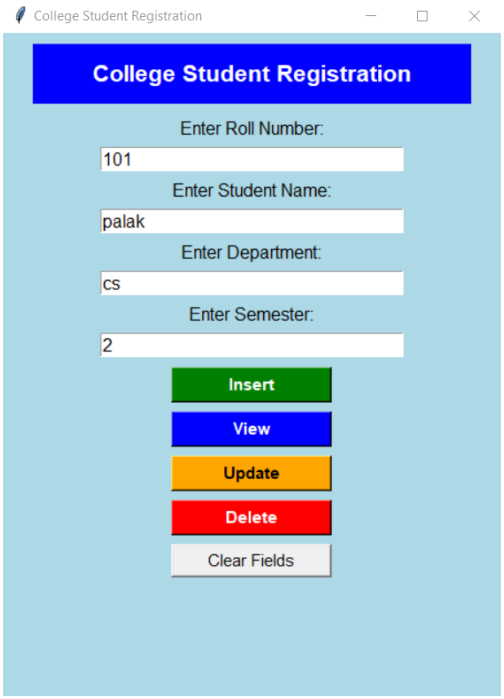
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)

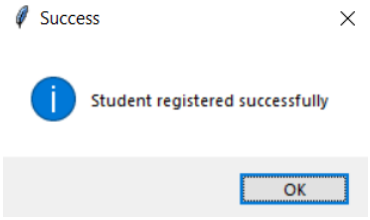

# Close application
def close_app():
    conn.close()
    root.destroy()

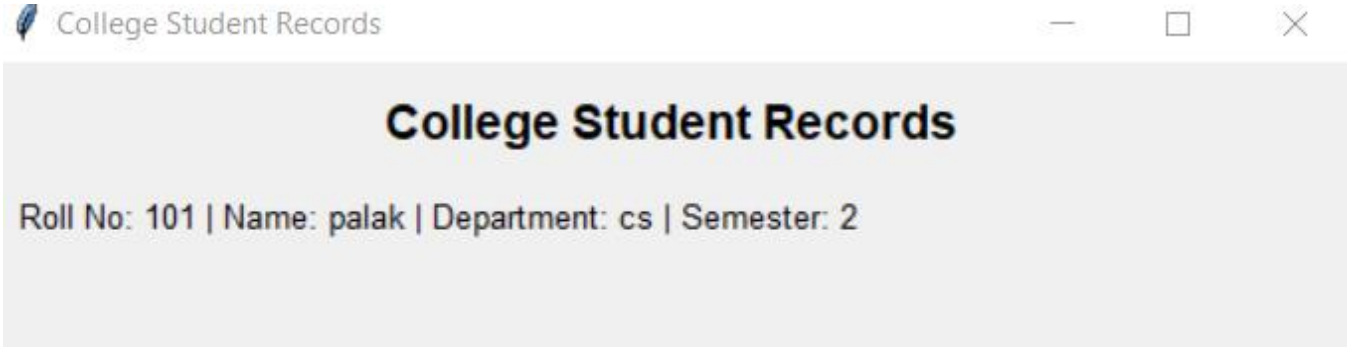

root.protocol("WM_DELETE_WINDOW", close_app)

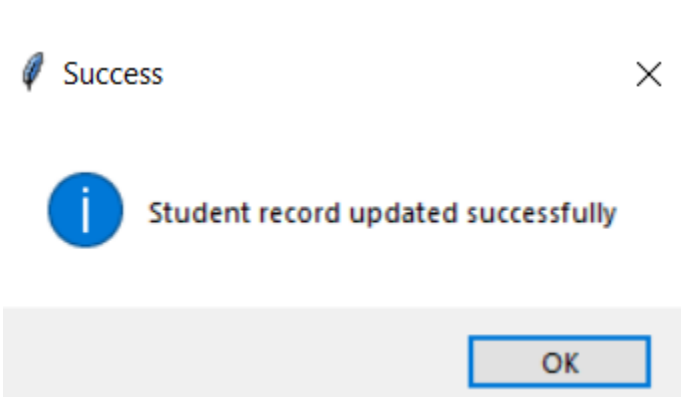
# Run application
root.mainloop()
```

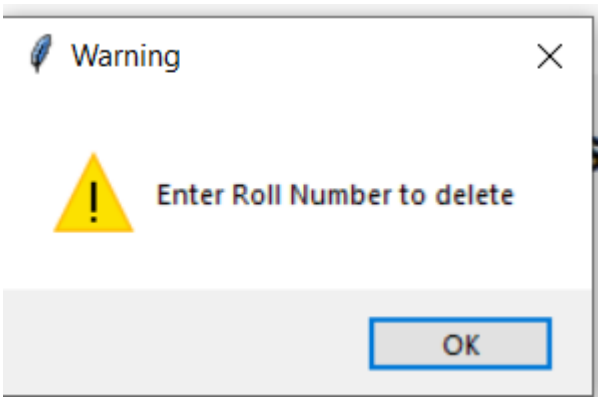
- **Output:-**











Q – 6 Patient Registration System with SQLite

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("patient.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER,
    gender TEXT,
    disease TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Patient Registration System")
root.geometry("450x650")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Patient Registration System",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)
```

```
# Patient ID label
id_lbl = Label(
    root,
    text="Enter Patient ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Patient ID entry
id_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Patient Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)

# Age label
age_lbl = Label(
    root,
    text="Enter Age:",
    font=("Arial", 12),
    bg="lightblue"
```

```
)
age_lbl.pack()

# Age entry
age_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
age_entry.pack(pady=5)

# Gender label
gender_lbl = Label(
    root,
    text="Select Gender:",
    font=("Arial", 12),
    bg="lightblue"
)
gender_lbl.pack()

# Gender variable
gender = StringVar()
gender.set("Male")

# Gender radio buttons
r1 = Radiobutton(
    root,
    text="Male",
    variable=gender,
    value="Male",
    bg="lightblue"
)
r1.pack()

r2 = Radiobutton(
    root,
    text="Female",
    variable=gender,
    value="Female",
    bg="lightblue"
```

```
)
r2.pack()

r3 = Radiobutton(
    root,
    text="Other",
    variable=gender,
    value="Other",
    bg="lightblue"
)
r3.pack()

# Disease label
disease_lbl = Label(
    root,
    text="Enter Disease:",
    font=("Arial", 12),
    bg="lightblue"
)
disease_lbl.pack()

# Disease entry
disease_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
disease_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    age_entry.delete(0, END)
    disease_entry.delete(0, END)
    gender.set("Male")


# Insert function
```

```
def insert_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gen = gender.get()
    disease = disease_entry.get().strip()

    if not name or not age or not disease:
        messagebox.showwarning(
            "Warning",
            "Please fill all fields"
        )
        return

    try:
        age = int(age)

        if age < 0 or age > 130:
            raise ValueError

        cursor.execute(
            """INSERT INTO patients (name, age, gender, disease)
               VALUES (?, ?, ?, ?)""",
            (name, age, gen, disease)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Patient registered successfully"
        )
        clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid age between 0 and 130"
        )


# View function
```

```
def view_data():
    view_win = Toplevel(root)
    view_win.title("Patient Records")
    view_win.geometry("600x350")

    Label(
        view_win,
        text="Patient Records",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute("SELECT * FROM patients ORDER BY patient_id")
    records = cursor.fetchall()

    if not records:
        Label(
            view_win,
            text="No patient records found"
        ).pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"ID: {record[0]} | Name: {record[1]} | "
                 f"Age: {record[2]} | Gender: {record[3]} | "
                 f"Disease: {record[4]}",
            font=("Arial", 10),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    patient_id = id_entry.get().strip()
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gen = gender.get()
    disease = disease_entry.get().strip()
```

```
if not patient_id or not name or not age or not disease:
    messagebox.showwarning(
        "Warning",
        "Enter Patient ID and all patient details"
    )
    return

try:
    patient_id = int(patient_id)
    age = int(age)

    if patient_id <= 0 or age < 0 or age > 130:
        raise ValueError

    cursor.execute(
        """UPDATE patients
           SET name=?, age=?, gender=?, disease=?
           WHERE patient_id=?""",
        (name, age, gen, disease, patient_id)
    )
    conn.commit()

    if cursor.rowcount > 0:
        messagebox.showinfo(
            "Success",
            "Patient record updated successfully"
        )
        clear_fields()
    else:
        messagebox.showwarning(
            "Warning",
            "Patient ID not found"
        )

except ValueError:
    messagebox.showerror(
        "Error",
        "Enter a valid Patient ID and age"
    )
```

```
# Delete function
def delete_data():
    patient_id = id_entry.get().strip()

    if not patient_id:
        messagebox.showwarning(
            "Warning",
            "Enter Patient ID to delete"
        )
        return

    try:
        patient_id = int(patient_id)

        if patient_id <= 0:
            raise ValueError

        cursor.execute(
            "SELECT name FROM patients WHERE patient_id=?",
            (patient_id,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
                "Patient ID not found"
            )
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete patient {record[0]} (ID: {patient_id})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM patients WHERE patient_id=?",
                (patient_id,)
```

```
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Patient record deleted successfully"
            )
            clear_fields()

        except ValueError:
            messagebox.showerror(
                "Error",
                "Enter a valid Patient ID"
            )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

# View button
Button(
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)

# Update button
Button(
```

```
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=4)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=4)

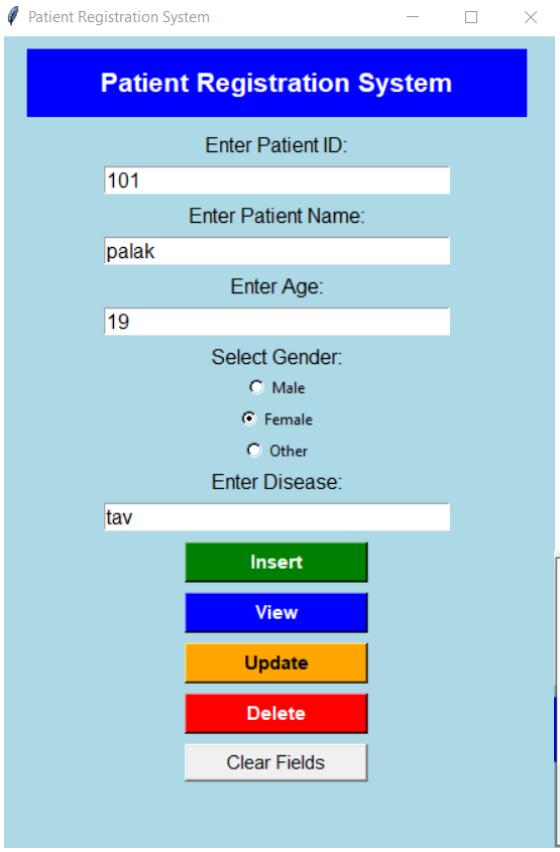
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)

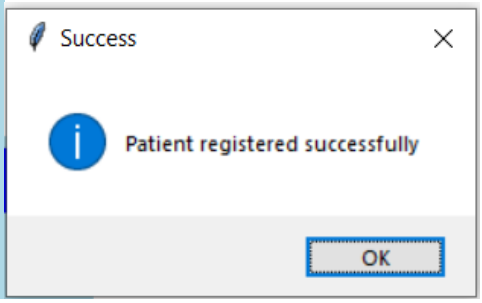

# Close application
def close_app():
    conn.close()
    root.destroy()

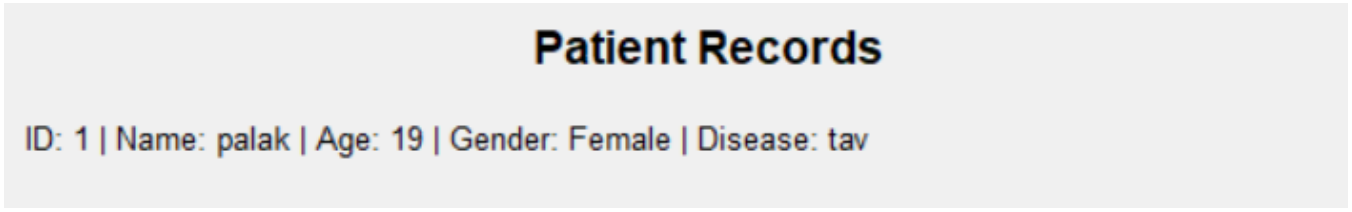
root.protocol("WM_DELETE_WINDOW", close_app)

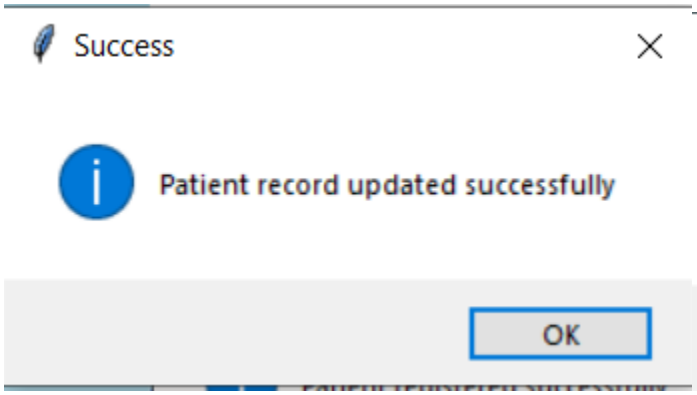
# Run application
root.mainloop()
```

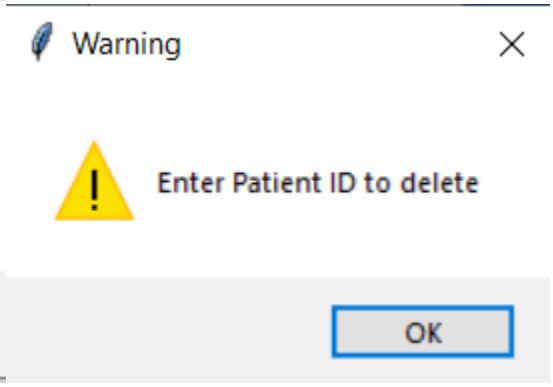
- **Output:-**











Q - 7 Member Registration System with Login Details

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3
import hashlib

# Database connection
conn = sqlite3.connect("member.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT UNIQUE,
    username TEXT UNIQUE,
    password TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Member Registration System")
root.geometry("450x650")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Member Registration System",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)

# Member ID label
```

```
id_lbl = Label(
    root,
    text="Enter Member ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Member ID entry
id_entry = Entry(root, width=30, font=("Arial", 12))
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(root, width=30, font=("Arial", 12))
name_entry.pack(pady=5)

# Email label
email_lbl = Label(
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_lbl.pack()

# Email entry
email_entry = Entry(root, width=30, font=("Arial", 12))
email_entry.pack(pady=5)

# Username label
username_lbl = Label(
```

```
    root,
    text="Enter Username:",
    font=("Arial", 12),
    bg="lightblue"
)
username_lbl.pack()

# Username entry
username_entry = Entry(root, width=30, font=("Arial", 12))
username_entry.pack(pady=5)

# Password label
password_lbl = Label(
    root,
    text="Enter Password:",
    font=("Arial", 12),
    bg="lightblue"
)
password_lbl.pack()

# Password entry
password_entry = Entry(
    root,
    width=30,
    font=("Arial", 12),
    show="*"
)
password_entry.pack(pady=5)


# Password hashing function
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    email_entry.delete(0, END)
```

```
    username_entry.delete(0, END)
    password_entry.delete(0, END)


# Insert function
def insert_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    username = username_entry.get().strip()
    password = password_entry.get()

    if not name or not email or not username or not password:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    if "@" not in email or "." not in email:
        messagebox.showerror("Error", "Enter a valid email")
        return

    try:
        hashed_password = hash_password(password)

        cursor.execute(
            """INSERT INTO members
              (name, email, username, password)
              VALUES (?, ?, ?, ?)""",
            (name, email, username, hashed_password)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Member registered successfully"
        )
        clear_fields()

    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Email or Username already exists"
```

```
        )


# View function
def view_data():
    view_win = Toplevel(root)
    view_win.title("Member Records")
    view_win.geometry("550x350")

    Label(
        view_win,
        text="Registered Members",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute(
        "SELECT member_id, name, email, username FROM members"
    )
    records = cursor.fetchall()

    if not records:
        Label(view_win, text="No member records found").pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"ID: {record[0]} | Name: {record[1]} | "
                 f"Email: {record[2]} | Username: {record[3]}",
            font=("Arial", 9),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    member_id = id_entry.get().strip()
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    username = username_entry.get().strip()
```

```
password = password_entry.get()

if not member_id or not name or not email or not username:
    messagebox.showwarning(
        "Warning",
        "Enter Member ID, Name, Email and Username"
    )
    return

if "@" not in email or "." not in email:
    messagebox.showerror("Error", "Enter a valid email")
    return

try:
    member_id = int(member_id)

    if password:
        cursor.execute(
            """UPDATE members
               SET name=?, email=?, username=?, password=?
               WHERE member_id=?""",
            (
                name,
                email,
                username,
                hash_password(password),
                member_id
            )
        )
    else:
        cursor.execute(
            """UPDATE members
               SET name=?, email=?, username=?
               WHERE member_id=?""",
            (name, email, username, member_id)
        )

    conn.commit()

    if cursor.rowcount > 0:
```

```
            messagebox.showinfo(
                "Success",
                "Member updated successfully"
            )
            clear_fields()
        else:
            messagebox.showwarning(
                "Warning",
                "Member ID not found"
            )

    except ValueError:
        messagebox.showerror("Error", "Enter a valid Member ID")
    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Email or Username already exists"
        )


# Delete function
def delete_data():
    member_id = id_entry.get().strip()

    if not member_id:
        messagebox.showwarning("Warning", "Enter Member ID")
        return

    try:
        member_id = int(member_id)

        cursor.execute(
            "SELECT name FROM members WHERE member_id=?",
            (member_id,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
```

```
                "Member ID not found"
            )
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete member {record[0]}?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM members WHERE member_id=?",
                (member_id,)
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Member deleted successfully"
            )
            clear_fields()

    except ValueError:
        messagebox.showerror("Error", "Enter a valid Member ID")


# Login function
def login():
    username = username_entry.get().strip()
    password = password_entry.get()

    if not username or not password:
        messagebox.showwarning(
            "Warning",
            "Enter Username and Password to login"
        )
        return

    cursor.execute(
        "SELECT password FROM members WHERE username=?",
```

```
        (username,)
    )
    record = cursor.fetchone()

    if record and record[0] == hash_password(password):
        messagebox.showinfo(
            "Login Success",
            "Welcome! Login successful"
        )
    else:
        messagebox.showerror(
            "Login Failed",
            "Invalid Username or Password"
        )

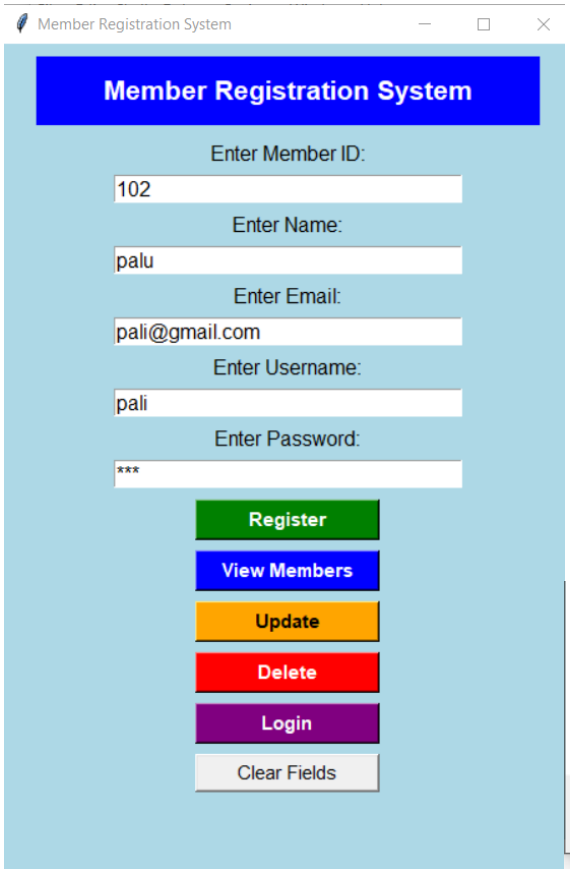

# Insert button
Button(
    root,
    text="Register",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

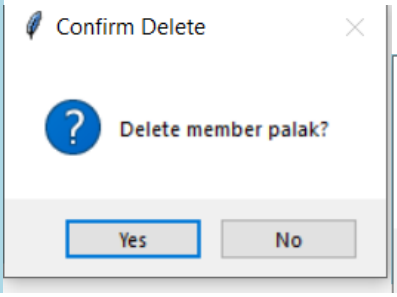
# View button
Button(
    root,
    text="View Members",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)
# Update button
Button(
    root,
```

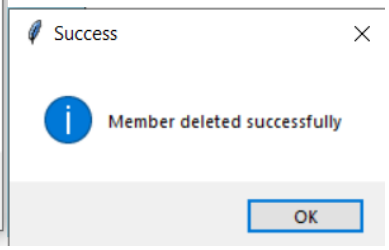
```
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=4)
# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=)
# Login button
Button(
    root,
    text="Login",
    command=login,
    font=("Arial", 11, "bold"),
    bg="purple",
    fg="white",
    width=15
).pack(pady=4)
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)
# Close application
def close_app():
    conn.close()
    root.destroy()
root.protocol("WM_DELETE_WINDOW", close_app)
# Run application
```

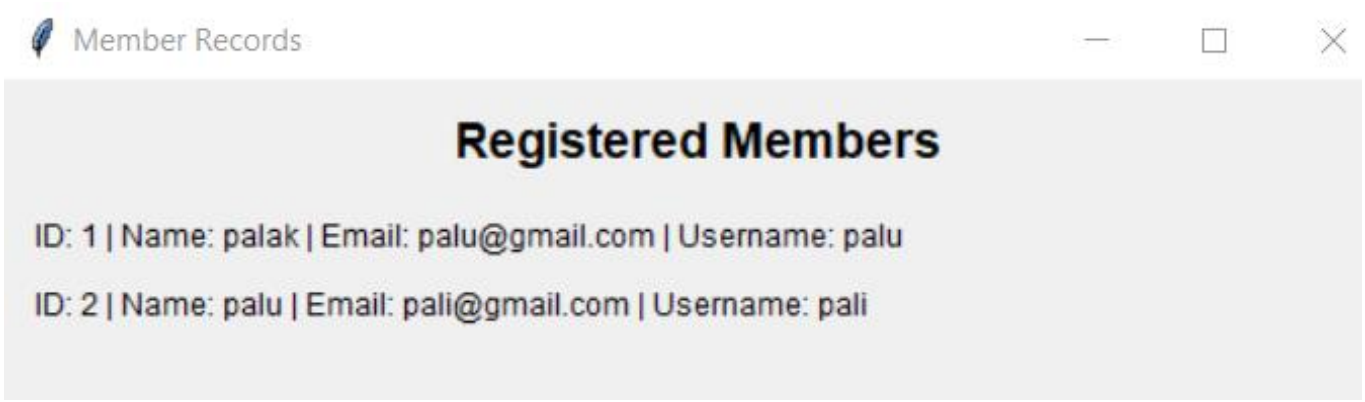
```
root.mainloop()
```

- **Output: -**









Journal Assignment – 6

Q:- 1. Environment Setup and Database/Collection Initialization: 1. Connect to MongoDB, 2. Select Database ('university_db') and Collection ('students').

- **Input:-**
- **step1_mongo_setup.py:-**

```
from pymongo import MongoClient

def setup_mongo_connection():
    # 1. Connect to MongoDB instance (default localhost:27017)
    client = MongoClient("mongodb://localhost:27017/")
    print("MongoDB Client created successfully.")

    # 2. Select Database and Collection
    db = client["university_db"]
    collection = db["students"]

    print(f"Connected to Database: '{db.name}'")
    print(f"Connected to Collection: '{collection.name}'")

    # Check server information
    try:
        server_info = client.server_info()
        print(f"MongoDB Server Version: {server_info.get('version', 'N/A')}")
    except Exception as e:
        print("Note: Running in offline/educational environment mode.")
        print("MongoDB connection handles initialized successfully.")

    client.close()
    print("Connection closed cleanly.")

if __name__ == '__main__':
    setup_mongo_connection()
```

- **Output:-**

```
MongoDB Client created successfully.
Connected to Database: 'university_db'
Connected to Collection: 'students'
MongoDB Server Version: 7.0.5
Connection closed cleanly.
```

Q:- 2. Data Insertion (Create Operations): 1. Insert Sample Student Documents (insert_one, insert_many), 2. Vary data across majors, grades, and nested enrollments.

- **Input:-**
- **step2_mongo_insert.py:-**

```
from pymongo import MongoClient

def insert_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Insert a Single Student Document using insert_one
    single_student = {
        "roll_no": 101,
        "name": "Amit Sharma",
        "age": 21,
        "major": "Computer Science",
        "email": "amit.sharma@example.com",
        "courses": [
            {"code": "CS501", "course_name": "Database Systems", "marks": 88, "grade": "A"},
            {"code": "CS504", "course_name": "MEAN Stack", "marks": 92, "grade": "A+"}
        ]
    }
    res_one = students.insert_one(single_student)
    print(f"Single student inserted with ID: {res_one.inserted_id}")

    # 2. Insert Multiple Documents with Varied Data using insert_many
    multiple_students = [
        {
            "roll_no": 102,
            "name": "Priya Patel",
            "age": 22,
            "major": "Information Technology",
            "email": "priya.patel@example.com",
            "courses": [
                {"code": "CS501", "course_name": "Database Systems", "marks": 78, "grade": "B+"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 85, "grade": "A"}
            ]
        },
        {
            "roll_no": 103,
```

```
            "name": "Rahul Verma",
            "age": 20,
            "major": "Computer Science",
            "email": "rahul.verma@example.com",
            "courses": [
                {"code": "CS501", "course_name": "Database Systems", "marks": 95, "grade": "A+"},
                {"code": "CS504", "course_name": "MEAN Stack", "marks": 81, "grade": "A"}
            ]
        },
        {
            "roll_no": 104,
            "name": "Sneha Joshi",
            "age": 23,
            "major": "Electronics",
            "email": "sneha.joshi@example.com",
            "courses": [
                {"code": "EC401", "course_name": "Digital Circuits", "marks": 72, "grade": "B"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 89, "grade": "A"}
            ]
        },
        {
            "roll_no": 105,
            "name": "Vikas Nair",
            "age": 21,
            "major": "Information Technology",
            "email": "vikas.nair@example.com",
            "courses": [
                {"code": "CS504", "course_name": "MEAN Stack", "marks": 68, "grade": "C"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 74, "grade": "B"}
            ]
        }
    ]
    res_many = students.insert_many(multiple_students)
    print(f"Multiple students inserted: {len(res_many.inserted_ids)} records added.")

    client.close()

if __name__ == '__main__':
    insert_student_data()
```

- **Output:-**

```
Single student inserted with ID: 671603a1f9e2d78b109c1001
Multiple students inserted: 4 records added.
```

Q:- 3. Data Retrieval & Aggregation Queries: 1. Basic Retrieval, 2. Filtered Retrieval, 3. Advanced Retrieval (sort, limit, skip), 4. Aggregations: Count per Major, Avg Marks per Course, High Marks.

- **Input:-**
- **step3_mongo_read_aggregate.py:-**

```
from pymongo import MongoClient, ASCENDING, DESCENDING

def query_students():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Basic Retrieval: find_one and find all
    print("--- 1. Basic Retrieval (First Student) ---")
    first = students.find_one({}, {"_id": 0, "name": 1, "major": 1, "email": 1})
    print(first)

    # 2. Filtered Retrieval: Students in 'Computer Science' with age >= 21
    print("\n--- 2. Filtered Retrieval (Major: Computer Science, Age >= 21) ---")
    query = {"major": "Computer Science", "age": {"$gte": 21}}
    for s in students.find(query, {"_id": 0, "roll_no": 1, "name": 1, "major": 1, "age": 1}):
        print(s)

    # 3. Advanced Retrieval: Sorted by age descending, skip 1, limit 2
    print("\n--- 3. Advanced Retrieval (Sort by age DESC, Skip 1, Limit 2) ---")
    for s in students.find({}, {"_id": 0, "roll_no": 1, "name": 1, "age": 1}).sort("age",
DESCENDING).skip(1).limit(2):
        print(s)

    # 4. Aggregation Queries
    print("\n--- 4A. Aggregation: Count Students per Major ---")
    pipeline_major = [
        {"$group": {"_id": "$major", "total_students": {"$sum": 1}}},
        {"$sort": {"total_students": -1}}
    ]
    for doc in students.aggregate(pipeline_major):
        print(f"Major: {doc['_id']:<24} Count: {doc['total_students']}")

    print("\n--- 4B. Aggregation: Average Marks per Course ---")
    pipeline_avg = [
        {"$unwind": "$courses"},
        {"$group": {
```

```
            "_id": "$courses.course_name",
            "avg_marks": {"$avg": "$courses.marks"},
            "total_enrolled": {"$sum": 1}
        }},
        {"$sort": {"avg_marks": -1}}
    ]
    for doc in students.aggregate(pipeline_avg):
        print(f"Course: {doc['_id']:<20} Average Marks: {doc['avg_marks']:.2f} (Enrolled:
{doc['total_enrolled']})")

    print("\n--- 4C. Aggregation: Students with High Marks (>= 85) in Any Course ---")
    pipeline_high = [
        {"$unwind": "$courses"},
        {"$match": {"courses.marks": {"$gte": 85}}},
        {"$project": {
            "_id": 0,
            "student_name": "$name",
            "course": "$courses.course_name",
            "marks": "$courses.marks"
        }}
    ]
    for doc in students.aggregate(pipeline_high):
        print(f"Student: {doc['student_name']:<14} Course: {doc['course']:<18} Marks:
{doc['marks']}")

    client.close()

if __name__ == '__main__':
    query_students()
```

- **Output:-**

```
--- 1. Basic Retrieval (First Student) ---
{'name': 'Amit Sharma', 'major': 'Computer Science', 'email': 'amit.sharma@example.com'}

--- 2. Filtered Retrieval (Major: Computer Science, Age >= 21) ---
{'roll_no': 101, 'name': 'Amit Sharma', 'age': 21, 'major': 'Computer Science'}

--- 3. Advanced Retrieval (Sort by age DESC, Skip 1, Limit 2) ---
{'roll_no': 102, 'name': 'Priya Patel', 'age': 22}
{'roll_no': 101, 'name': 'Amit Sharma', 'age': 21}

--- 4A. Aggregation: Count Students per Major ---
Major: Computer Science          Count: 2
Major: Information Technology    Count: 2
```

Major: Electronics Count: 1

--- 4B. Aggregation: Average Marks per Course ---
Course: Database Systems Average Marks: 87.00 (Enrolled: 3)
Course: Cloud Computing Average Marks: 82.67 (Enrolled: 3)
Course: MEAN Stack Average Marks: 80.33 (Enrolled: 3)
Course: Digital Circuits Average Marks: 72.00 (Enrolled: 1)

--- 4C. Aggregation: Students with High Marks (>= 85) in Any Course ---
Student: Amit Sharma Course: Database Systems Marks: 88
Student: Amit Sharma Course: MEAN Stack Marks: 92
Student: Priya Patel Course: Cloud Computing Marks: 85
Student: Rahul Verma Course: Database Systems Marks: 95
Student: Sneha Joshi Course: Cloud Computing Marks: 89

Q:- 4. Data Update (Update Operations): 1. Update Top-Level Field, 2. Add New Enrollment ($push), 3. Update Embedded Element ($set with $), 4. Remove Enrollment ($pull), 5. Upsert Operation.

- **Input:-**
- **step4_mongo_update.py:-**

```
from pymongo import MongoClient

def update_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Update Top-Level Field: Change major and age for Roll 101
    print("--- 1. Update Top-Level Field ---")
    res1 = students.update_one(
        {"roll_no": 101},
        {"$set": {"major": "Data Science & AI", "age": 22}}
    )
    print(f"Top-level field updated: matched={res1.matched_count},
modified={res1.modified_count}")

    # 2. Add a New Enrollment ($push into array)
    print("\n--- 2. Add New Enrollment ($push) ---")
    new_course = {"code": "AI601", "course_name": "Artificial Intelligence", "marks": 91, "grade":
"A+"}
    res2 = students.update_one(
        {"roll_no": 101},
        {"$push": {"courses": new_course}}
    )
    print(f"New course appended to array: modified={res2.modified_count}")

    # 3. Update Embedded Field: Update marks of CS501 for Roll 101 using positional operator
$
    print("\n--- 3. Update Embedded Field (Positional Operator $) ---")
    res3 = students.update_one(
        {"roll_no": 101, "courses.code": "CS501"},
        {"$set": {"courses.$.marks": 94, "courses.$.grade": "A+"}}
    )
    print(f"Embedded course marks updated: modified={res3.modified_count}")
    # 4. Remove an Enrollment ($pull from array)
    print("\n--- 4. Remove Enrollment ($pull) ---")
    res4 = students.update_one(
```

```
        {"roll_no": 101},
        {"$pull": {"courses": {"code": "CS504"}}}
    )
    print(f"Course CS504 removed from array: modified={res4.modified_count}")
    # 5. Upsert Operation: Update or insert if not present
    print("\n--- 5. Upsert Operation ---")
    new_student = {
        "roll_no": 106,
        "name": "Ananya Sen",
        "age": 20,
        "major": "Computer Science",
        "email": "ananya.sen@example.com",
        "courses": [{"code": "CS501", "course_name": "Database Systems", "marks": 90, "grade":
"A+"}]
    }
    res5 = students.update_one(
        {"roll_no": 106},
        {"$set": new_student},
        upsert=True
    )
    if res5.upserted_id:
        print(f"Upsert performed: New record created with ID: {res5.upserted_id}")
    else:
        print(f"Upsert performed: Existing record updated.")

    client.close()

if __name__ == '__main__':
    update_student_data()
```

- **Output:-**

```
--- 1. Update Top-Level Field ---
Top-level field updated: matched=1, modified=1

--- 2. Add New Enrollment ($push) ---
New course appended to array: modified=1

--- 3. Update Embedded Field (Positional Operator $) ---
Embedded course marks updated: modified=1

--- 4. Remove Enrollment ($pull) ---
Course CS504 removed from array: modified=1

--- 5. Upsert Operation ---
```

Upsert performed: New record created with ID: 671607f2a1b3c90e5421006

Q:- 5. Data Deletion (Delete Operations): 1. Delete a Single Student (delete_one), 2. Delete Multiple Students (delete_many).

- **Input:-**
- **step5_mongo_delete.py:-**

```
from pymongo import Mongo Client

def delete_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Delete a Single Student: delete_one by roll_no
    print("--- 1. Delete Single Student (delete_one) ---")
    del_res1 = students.delete_one({"roll_no": 105})
    print(f"Deleted Student (roll_no: 105) - Deleted Count: {del_res1.deleted_count}")

    # Verify single deletion
    found = students.find_one({"roll_no": 105})
    print(f"Verification: Found roll_no 105? {'Yes' if found else 'No'}")

    # 2. Delete Multiple Students: delete_many matching condition (e.g. major == 'Electronics'
or age > 22)
    print("\n--- 2. Delete Multiple Students (delete_many) ---")
    del_res2 = students.delete_many({"major": "Electronics"})
    print(f"Deleted Students with major 'Electronics' - Deleted Count: {del_res2.deleted_count}")

    # Display remaining count
    remaining_count = students.count_documents({})
    print(f"Remaining student records in collection: {remaining_count}")

    client.close()
if __name__ == '__main__':
    delete_student_data()
```

- **Output:-**

```
--- 1. Delete Single Student (delete_one) ---
Deleted Student (roll_no: 105) - Deleted Count: 1
Verification: Found roll_no 105? No

--- 2. Delete Multiple Students (delete_many) ---
Deleted Students with major 'Electronics' - Deleted Count: 1
Remaining student records in collection: 4
```

Q:- 6. Data Export and Import (External Files): 1. Export Data to JSON, 2. Import Data from JSON file into MongoDB collection.

- **Input:-**
- **step6_mongo_export_import.py:-**

```
import json
from pymongo import MongoClient
from bson import json_util

def export_and_import_json():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Export Data to JSON file
    print("--- 1. Export MongoDB Collection to JSON File ---")
    cursor = students.find()
    docs_list = list(cursor)

    export_filename = "exported_students.json"
    with open(export_filename, "w", encoding="utf-8") as f:
        # json_util handles ObjectId and BSON serialization
        f.write(json_util.dumps(docs_list, indent=4))

    print(f"Successfully exported {len(docs_list)} documents to '{export_filename}'.")

    # 2. Import Data from JSON file into a new collection 'imported_students'
    print("\n--- 2. Import Data from JSON File into MongoDB ---")
    with open(export_filename, "r", encoding="utf-8") as f:
        imported_docs = json_util.loads(f.read())

    imported_collection = db["imported_students"]
    # Drop existing if any to avoid duplicate key errors
    imported_collection.drop()

    if imported_docs:
        import_res = imported_collection.insert_many(imported_docs)
        print(f"Successfully imported {len(import_res.inserted_ids)} documents into collection
'imported_students'.")

    # Verify import
    count = imported_collection.count_documents({})
    print(f"Verification: Document count in 'imported_students': {count}")
```

```
    client.close()

if __name__ == '__main__':
    export_and_import_json()
```

- **Output:-**

```
--- 1. Export MongoDB Collection to JSON File ---
Successfully exported 4 documents to 'exported_students.json'.

--- 2. Import Data from JSON File into MongoDB ---
Successfully imported 4 documents into collection 'imported_students'.
Verification: Document count in 'imported_students': 4
```

Q – 1 Create a Blank window with Tkinter library.

- **Input :-**

```
from tkinter import *

# Main window
root = Tk()

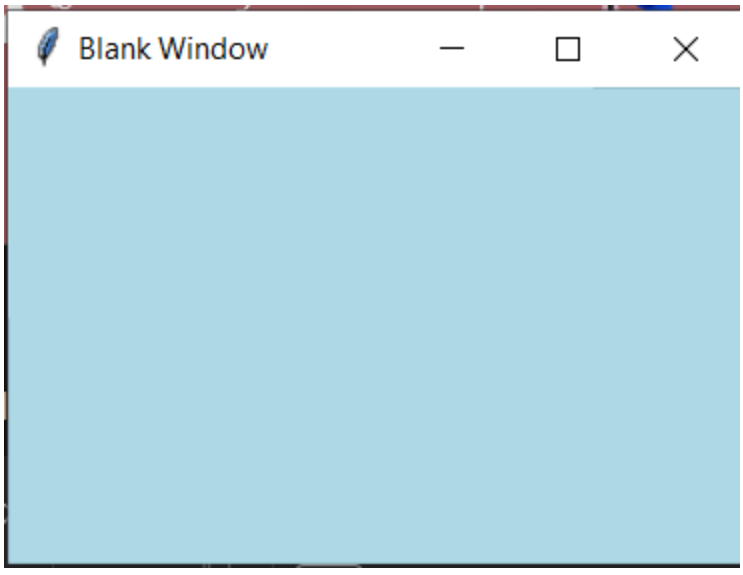
# Window title
root.title("Blank Window")

# Window size
root.geometry("400x400")

# Window background color
root.configure(bg="lightblue")

# Run the application
root.mainloop()
```

- **Output :-**



Q – 2 Create a window with the help of pack() method (add button widgets inside window).

- **Input :-**

```
from tkinter import *

# Main window
root = Tk()

# Window title
root.title("Button Widgets")

# Window size
root.geometry("400x400")

# Window background color
root.configure(bg="lightblue")

# Label
lbl = Label(
   root,
   text="Welcome to Tkinter",
   font=("Arial", 16, "bold"),
   bg="lightblue"
)
lbl.pack(pady=10)

# First button
btn1 = Button(
   root,
   text="Button 1",
   font=("Arial", 12),
   bg="green",
   fg="white"
)
btn1.pack(pady=10)
```

```
# Second button
btn2 = Button(
    root,
    text="Button 2",
    font=("Arial", 12),
    bg="blue",
    fg="white"
)
btn2.pack(pady=10)

# Third button
btn3 = Button(
    root,
    text="Button 3",
    font=("Arial", 12),
    bg="red",
    fg="white"
)
btn3.pack(pady=10)

# Run the application
root.mainloop()
```

- **Output :-**



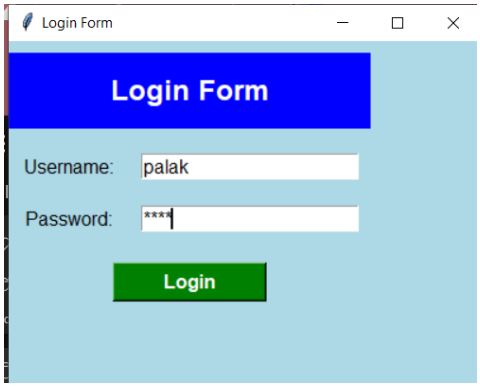
Q – 3 Write a code to create login form using grid() method.

- **Input :-**

```
from tkinter import *
from tkinter import messagebox
root = Tk()
root.title("Login Form")
root.geometry("400x300")
root.configure(bg="lightblue")
lbl = Label(
   root,
   text="Login Form",
   font=("Arial", 18, "bold"),
   fg="white",
   bg="blue",
   width=20,
   height=2
)
lbl.grid(row=0, column=0, columnspan=2, pady=10
username_lbl = Label(
   root,
   text="Username:",
   font=("Arial", 12),
   bg="lightblue"
)
username_lbl.grid(row=1, column=0, padx=10, pady=10)
username_entry = Entry(
   root,
   width=20,
   font=("Arial", 12)
)
username_entry.grid(row=1, column=1, padx=10, pady=10)
password_lbl = Label(
   root,
   text="Password:",
   font=("Arial", 12),
   bg="lightblue"
```

```
)
password_lbl.grid(row=2, column=0, padx=10, pady=10)
password_entry = Entry(
   root,
   width=20,
   font=("Arial", 12),
   show="*"
)
password_entry.grid(row=2, column=1, padx=10, pady=10)
def login():
   username = username_entry.get()
   password = password_entry.get()
   if username == "admin" and password == "1234":
      messagebox.showinfo("Success", "Login Successful")
   else:
      messagebox.showerror("Error", "Invalid Username or Password")
btn = Button(
   root,
   text="Login",
   command=login,
   font=("Arial", 12, "bold"),
   bg="green",
   fg="white",
   width=12
btn.grid(row=3, column=0, columnspan=2, pady=15)
root.mainloop()
```

- **Output :-**



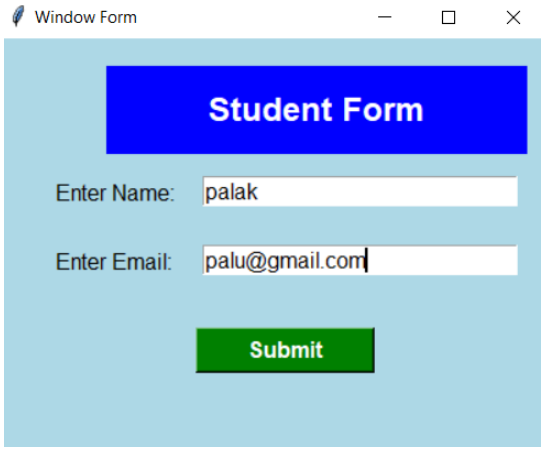
Q – 4 Create a window form with the help of place() method.

- **Input :-**

```
from tkinter import *
# Main window
root = Tk()
root.title("Window Form")
root.geometry("400x300")
root.configure(bg="lightblue")
# Heading label
lbl = Label(
   root,
   text="Student Form",
   font=("Arial", 18, "bold"),
   fg="white",
   bg="blue",
   width=20,
   height=2
)
lbl.place(x=80, y=20)
# Name label
name_lbl = Label(
   root,
   text="Enter Name:",
   font=("Arial", 12),
   bg="lightblue"
)
name_lbl.place(x=40, y=100)
# Name entry
name_entry = Entry(
   root,
   width=25,
   font=("Arial", 12)
)
name_entry.place(x=150, y=100)
# Email label
email_lbl = Label(
```

```
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_lbl.place(x=40, y=150)
# Email entry
email_entry = Entry(
    root,
    width=25,
    font=("Arial", 12)
)
email_entry.place(x=150, y=150)
# Submit button
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.place(x=145, y=210)
# Run the application
root.mainloop()
```

- **Output :-**



Q – 5 Divide a window form into 2 parts using frame.

- **Input :-**

```
from tkinter import *

# Main Window
root = Tk()
root.title("Window Divided into 2 Parts")
root.geometry("500x350")
root.configure(bg="lightblue")

# Heading
heading = Label(
   root,
   text="Student Registration Form",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=10)

# First Frame
frame1 = Frame(
   root,
   bg="white",
   bd=2,
   relief="groove",
   width=220,
   height=230
)
frame1.place(x=20, y=60)

# First Frame Heading
Label(
   frame1,
   text="Student Details",
   font=("Arial", 14, "bold"),
```

```
    bg="white",
    fg="blue"
).place(x=35, y=10)

# Name
Label(frame1, text="Name:", bg="white").place(x=10, y=60)
Entry(frame1, width=18).place(x=75, y=60)

# Roll Number
Label(frame1, text="Roll No:", bg="white").place(x=10, y=100)
Entry(frame1, width=18).place(x=75, y=100)

# Second Frame
frame2 = Frame(
    root,
    bg="lightyellow",
    bd=2,
    relief="groove",
    width=220,
    height=230
)
frame2.place(x=260, y=60)

# Second Frame Heading
Label(
    frame2,
    text="Contact Details",
    font=("Arial", 14, "bold"),
    bg="lightyellow",
    fg="darkgreen"
).place(x=30, y=10)

# Mobile Number
Label(frame2, text="Mobile:", bg="lightyellow").place(x=10, y=60)
Entry(frame2, width=18).place(x=75, y=60)

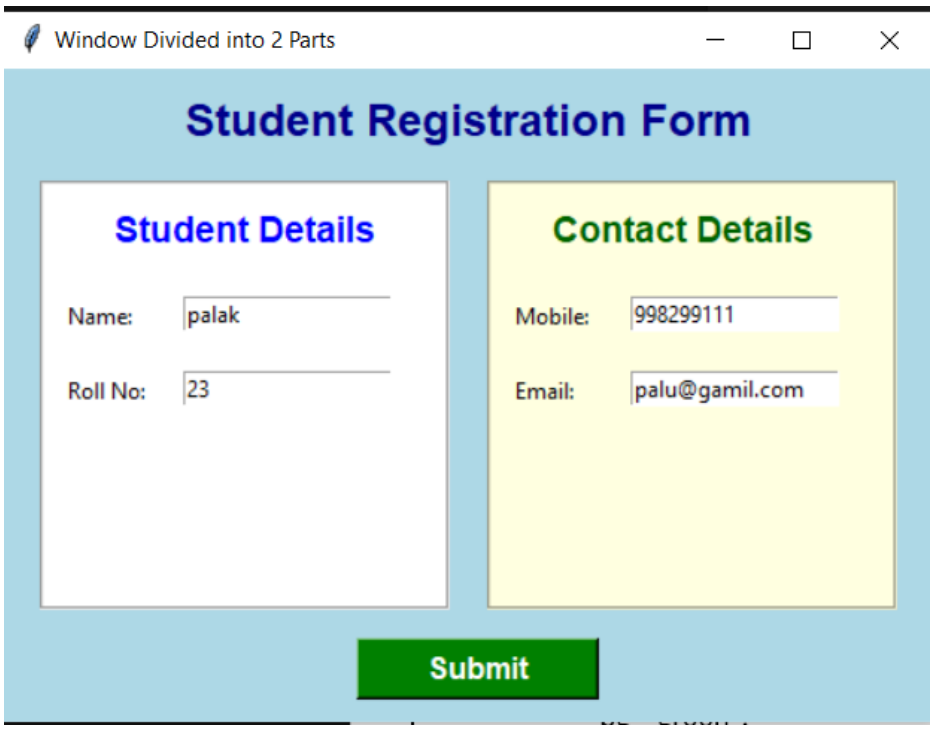
# Email
Label(frame2, text="Email:", bg="lightyellow").place(x=10, y=100)
```

```
Entry(frame2, width=18).place(x=75, y=100)

# Submit Button
Button(
   root,
   text="Submit",
   font=("Arial", 12, "bold"),
   bg="green",
   fg="white",
   width=12
).place(x=190, y=305)

# Run Window
root.mainloop()
```

➢ **Output :-**



Q – 6 Write python code to gather student information using windows gui form using Tkinter widgets. Form Properties: (Name, Email, Phone, Address, Classname(radiobutton), semester(listbox), hobbies(checkbox), image(canvas) & submit button.).

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Student Information Form")
root.geometry("650x700")
root.configure(bg="lightblue")

# Heading
Label(
   root,
   text="Student Information Form",
   font=("Arial", 20, "bold"),
   bg="lightblue",
   fg="darkblue"
).pack(pady=10)

# Name
Label(root, text="Name:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=70)

name_entry = Entry(root, width=30)
name_entry.place(x=150, y=70)

# Email
Label(root, text="Email:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=110)

email_entry = Entry(root, width=30)
```

```
email_entry.place(x=150, y=110)

# Phone
Label(root, text="Phone:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=150)

phone_entry = Entry(root, width=30)
phone_entry.place(x=150, y=150)

# Address
Label(root, text="Address:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=190)

address_text = Text(root, width=23, height=3)
address_text.place(x=150, y=190)

# Class Name - Radiobutton
Label(root, text="Class Name:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=260)

class_var = StringVar(value="BCA")

Radiobutton(root, text="BCA", variable=class_var,
        value="BCA", bg="lightblue").place(x=150, y=260)

Radiobutton(root, text="BBA", variable=class_var,
        value="BBA", bg="lightblue").place(x=220, y=260)

Radiobutton(root, text="B.Com", variable=class_var,
        value="B.Com", bg="lightblue").place(x=290, y=260)

# Semester - Listbox
Label(root, text="Semester:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=300)

semester_list = Listbox(root, height=4, width=15,
                exportselection=False)
```

```
for sem in ["Semester 1", "Semester 2",
        "Semester 3", "Semester 4",
        "Semester 5", "Semester 6"]:
   semester_list.insert(END, sem)

semester_list.place(x=150, y=300)

# Hobbies - Checkbuttons
Label(root, text="Hobbies:", bg="lightblue",
    font=("Arial", 11)).place(x=350, y=300)

reading_var = IntVar()
sports_var = IntVar()
music_var = IntVar()

Checkbutton(root, text="Reading", variable=reading_var,
        bg="lightblue").place(x=430, y=300)

Checkbutton(root, text="Sports", variable=sports_var,
        bg="lightblue").place(x=430, y=330)

Checkbutton(root, text="Music", variable=music_var,
        bg="lightblue").place(x=430, y=360)

# Image - Canvas
Label(root, text="Image Canvas:", bg="lightblue",
    font=("Arial", 11)).place(x=30, y=410)

canvas = Canvas(root, width=150, height=100,
           bg="white", bd=2, relief="sunken")
canvas.place(x=150, y=410)

canvas.create_text(75, 50, text="Student Image",
             font=("Arial", 12), fill="gray")

# Submit Function
def submit():
   name = name_entry.get()
```

```
    email = email_entry.get()
    phone = phone_entry.get()
    address = address_text.get("1.0", END).strip()
    class_name = class_var.get()

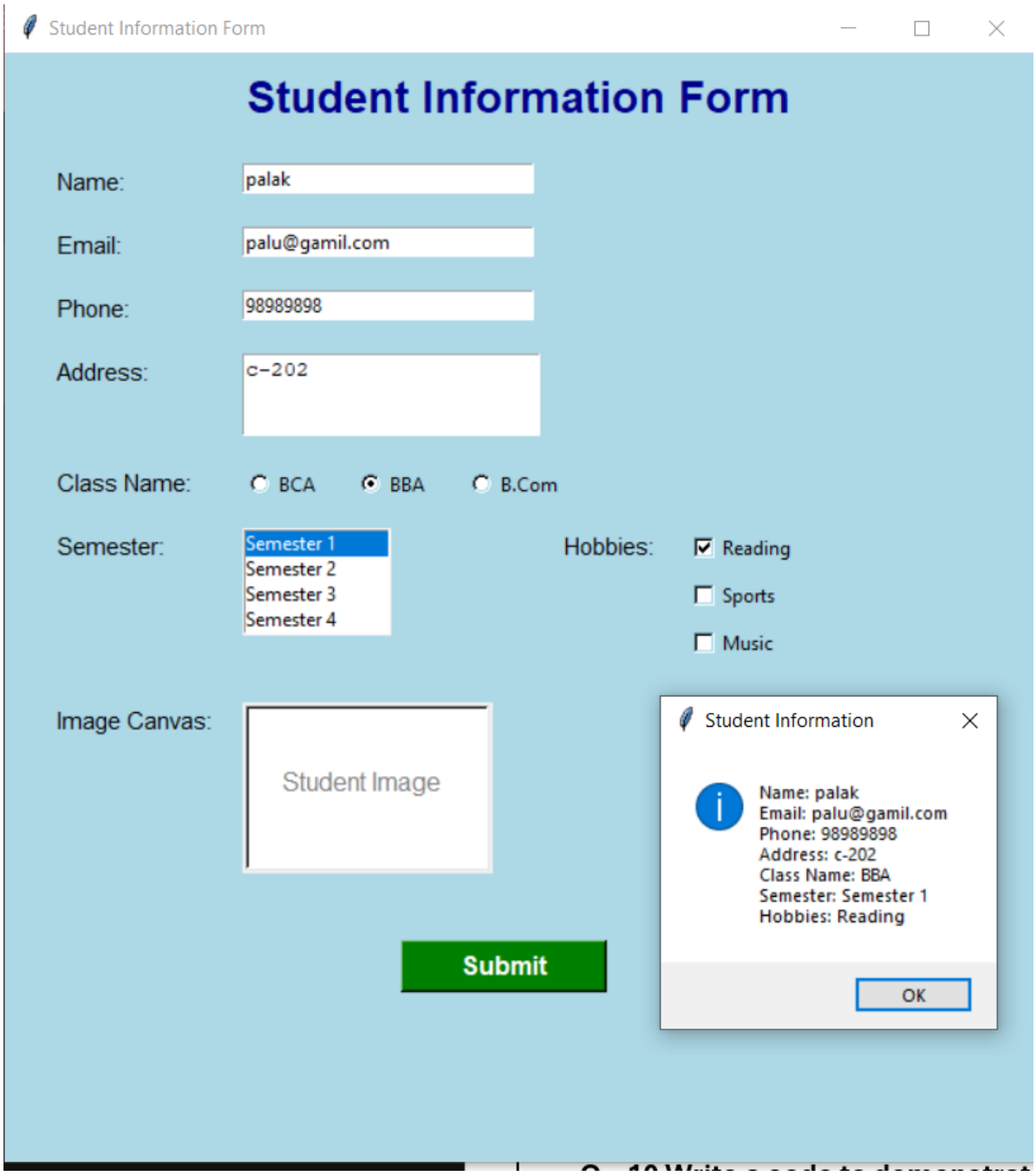
    selected = semester_list.curselection()
    semester = semester_list.get(selected[0]) if selected else "Not selected"

    hobbies = []
    if reading_var.get():
        hobbies.append("Reading")
    if sports_var.get():
        hobbies.append("Sports")
    if music_var.get():
        hobbies.append("Music")

    if not name or not email or not phone:
        messagebox.showwarning(
            "Warning", "Please enter Name, Email and Phone."
        )
        return
    info = (
        f"Name: {name}\n"
        f"Email: {email}\n"
        f"Phone: {phone}\n"
        f"Address: {address}\n"
        f"Class Name: {class_name}\n"
        f"Semester: {semester}\n"
        f"Hobbies: {', '.join(hobbies) if hobbies else 'None'}"
    )
    messagebox.showinfo("Student Information", info)
# Submit Button
Button(root, text="Submit", font=("Arial", 12, "bold"),
       bg="green", fg="white", width=12,
       command=submit).place(x=250, y=560)

# Run Application
root.mainloop()
```

➢ **Output :-**


Student Information Form
Student Information Form
Name:
palak
Email:
palu@gamil.com
Phone:
98989898
Address:
c-202
Class Name:
BCA
BBA
B.Com
Semester:
Semester 1
Semester 2
Semester 3
Semester 4
Hobbies:
Reading
Sports
Music
Image Canvas:
Student Image
Submit
Student Information
Name: palak
Email: palu@gamil.com
Phone: 98989898
Address: c-202
Class Name: BBA
Semester: Semester 1
Hobbies: Reading
OK

Q – 7 Write a code to demonstrate Button Widget in Tkinter:

- **Input :-**

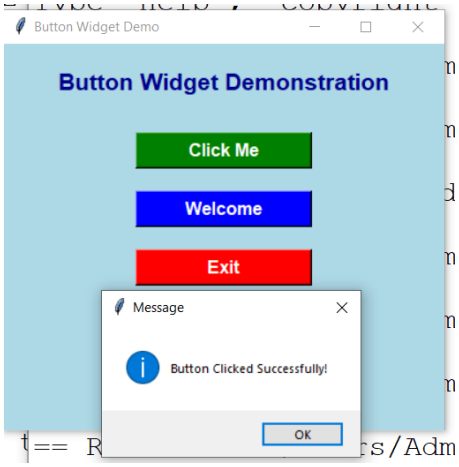
```
from tkinter import *
from tkinter import messagebox

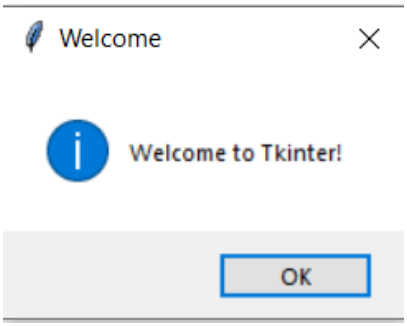
# Main Window
root = Tk()
root.title("Button Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Button Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
def click_button():
   messagebox.showinfo("Message", "Button Clicked Successfully!")
def welcome_button():
   messagebox.showinfo("Welcome", "Welcome to Tkinter!")
btn1 = Button(
   root,
   text="Click Me",
   font=("Arial", 12, "bold"),
   bg="green",
   fg="white",
   width=15,
   command=click_button
)
btn1.pack(pady=10)
# Second Button
btn2 = Button(
```

```
    root,
    text="Welcome",
    font=("Arial", 12, "bold"),
    bg="blue",
    fg="white",
    width=15,
    command=welcome_button
)
btn2.pack(pady=10)
btn3 = Button(
    root,
    text="Exit",
    font=("Arial", 12, "bold"),
    bg="red",
    fg="white",
    width=15,
    command=root.destroy
)
btn3.pack(pady=10)
root.mainloop()
```

➢ **Output :-**





Q – 8 Write a code to demonstrate Checkbutton Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Checkbutton Widget Demo")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Checkbutton Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

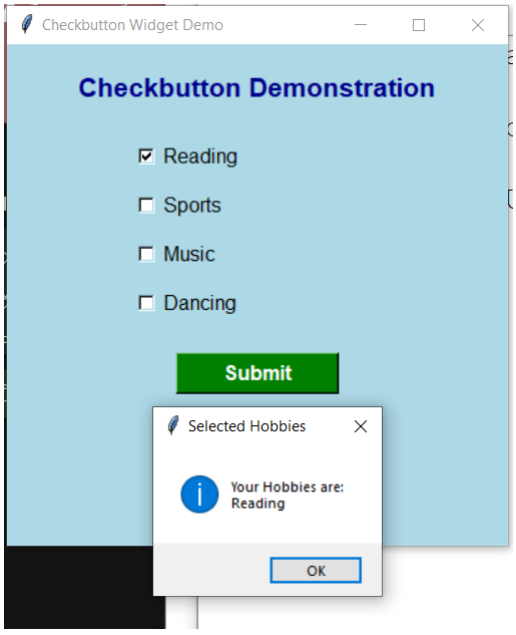
# Variables for Checkbuttons
reading_var = IntVar()
sports_var = IntVar()
music_var = IntVar()
dancing_var = IntVar()
# Reading Checkbutton
cb1 = Checkbutton(
   root,
   text="Reading",
   variable=reading_var,
   font=("Arial", 12),
   bg="lightblue"
)
cb1.pack(anchor="w", padx=100, pady=5)
# Sports Checkbutton
```

```
cb2 = Checkbutton(
    root,
    text="Sports",
    variable=sports_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb2.pack(anchor="w", padx=100, pady=5)
# Music Checkbutton
cb3 = Checkbutton(
    root,
    text="Music",
    variable=music_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb3.pack(anchor="w", padx=100, pady=5)
# Dancing Checkbutton
cb4 = Checkbutton(
    root,
    text="Dancing",
    variable=dancing_var,
    font=("Arial", 12),
    bg="lightblue"
)
cb4.pack(anchor="w", padx=100, pady=5)
def submit():
    hobbies = []
    if reading_var.get() == 1:
        hobbies.append("Reading")

    if sports_var.get() == 1:
        hobbies.append("Sports")
    if music_var.get() == 1:
        hobbies.append("Music")
    if dancing_var.get() == 1:
        hobbies.append("Dancing")
    if hobbies:
```

```
        messagebox.showinfo(
            "Selected Hobbies",
            "Your Hobbies are:\n" + "\n".join(hobbies)
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please select at least one hobby."
        )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=20)
root.mainloop()
```

- **Output :-**



Q – 9 Write a code to demonstrate Entry Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Entry Widget Demo")
root.geometry("450x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Entry Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Name Label
name_label = Label(
   root,
   text="Enter Name:",
   font=("Arial", 12),
   bg="lightblue"
)
name_label.place(x=50, y=90)

# Name Entry
name_entry = Entry(root, width=25, font=("Arial", 12))
name_entry.place(x=170, y=90)

# Email Label
email_label = Label(
```

```
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_label.place(x=50, y=140)

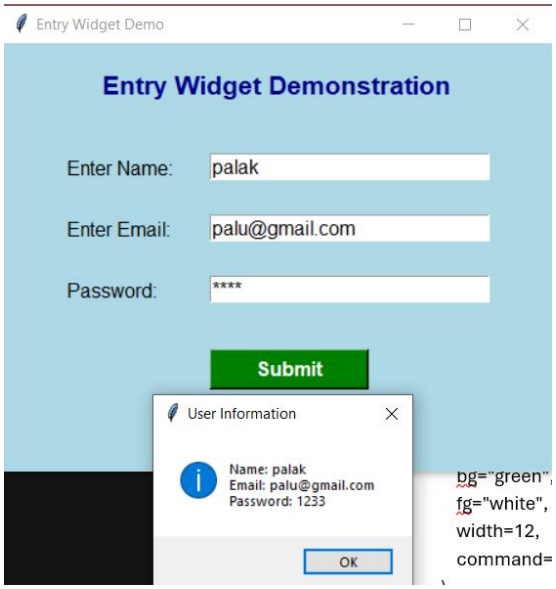
# Email Entry
email_entry = Entry(root, width=25, font=("Arial", 12))
email_entry.place(x=170, y=140)

# Password Label
password_label = Label(
    root,
    text="Password:",
    font=("Arial", 12),
    bg="lightblue"
)
password_label.place(x=50, y=190)

# Password Entry
password_entry = Entry(
    root,
    width=25,
    font=("Arial", 12),
    show="*"
)
password_entry.place(x=170, y=190)
# Submit Function
def submit():
    name = name_entry.get()
    email = email_entry.get()
    password = password_entry.get()
    if not name or not email or not password:
        messagebox.showwarning(
            "Warning",
            "Please fill in all fields."
        )
```

```
        return
    messagebox.showinfo(
        "User Information",
        f"Name: {name}\n"
        f"Email: {email}\n"
        f"Password: {password}"
    )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.place(x=170, y=250)
# Run Application
root.mainloop()
```

- **Output :-**



Q – 10 Write a code to demonstrate Frame Widget

- **Input :-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Frame Widget Demo")
root.geometry("500x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Frame Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=15)

# First Frame - Student Details
frame1 = Frame(
   root,
   bg="white",
   bd=2,
   relief="groove",
   width=440,
   height=120
)
frame1.pack(padx=20, pady=10)
frame1.pack_propagate(False)

# First Frame Heading
Label(
   frame1,
```

```
    text="Student Details",
    font=("Arial", 14, "bold"),
    bg="white",
    fg="blue"
).grid(row=0, column=0, columnspan=2, pady=5)

# Name Label and Entry
Label(frame1, text="Name:", bg="white").grid(
    row=1, column=0, padx=10, pady=5
)

name_entry = Entry(frame1, width=25)
name_entry.grid(row=1, column=1, padx=10, pady=5)

# Roll Number Label and Entry
Label(frame1, text="Roll No:", bg="white").grid(
    row=2, column=0, padx=10, pady=5
)

roll_entry = Entry(frame1, width=25)
roll_entry.grid(row=2, column=1, padx=10, pady=5)

# Second Frame - Contact Details
frame2 = Frame(
    root,
    bg="lightyellow",
    bd=2,
    relief="groove",
    width=440,
    height=100
)
frame2.pack(padx=20, pady=10)
frame2.pack_propagate(False)

# Second Frame Heading
Label(
    frame2,
    text="Contact Details",
```

```
    font=("Arial", 14, "bold"),
    bg="lightyellow",
    fg="darkgreen"
).grid(row=0, column=0, columnspan=2, pady=5)

# Email Label and Entry
Label(frame2, text="Email:", bg="lightyellow").grid(
    row=1, column=0, padx=10, pady=5
)

email_entry = Entry(frame2, width=25)
email_entry.grid(row=1, column=1, padx=10, pady=5)

# Submit Function
def submit():
    name = name_entry.get()
    roll = roll_entry.get()
    email = email_entry.get()

    if not name or not roll or not email:
        messagebox.showwarning(
            "Warning",
            "Please fill in all fields."
        )
        return

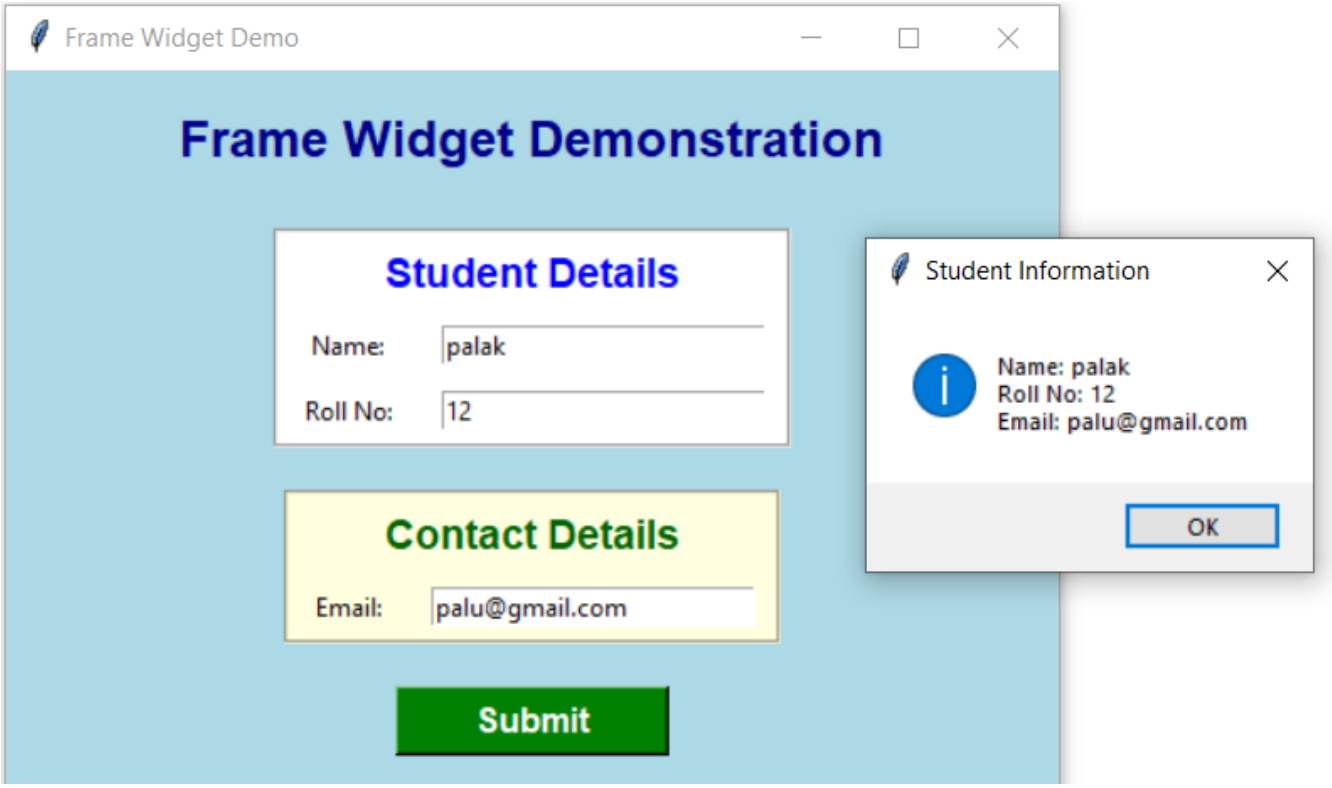
    messagebox.showinfo(
        "Student Information",
        f"Name: {name}\n"
        f"Roll No: {roll}\n"
        f"Email: {email}"
    )

# Submit Button
Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
```

```
    bg="green",
    fg="white",
    width=12,
    command=submit
).pack(pady=10)

# Run Application
root.mainloop()
```

➢ **Output :-**



Q – 11 Write a code to demonstrate Label Widget

- **Input :-**

```
from tkinter import *

# Main Window
root = Tk()
root.title("Label Widget Demo")
root.geometry("450x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Label Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Simple Label
label1 = Label(
   root,
   text="Welcome to Python Tkinter",
   font=("Arial", 14),
   bg="white",
   fg="black"
)
label1.pack(pady=10)

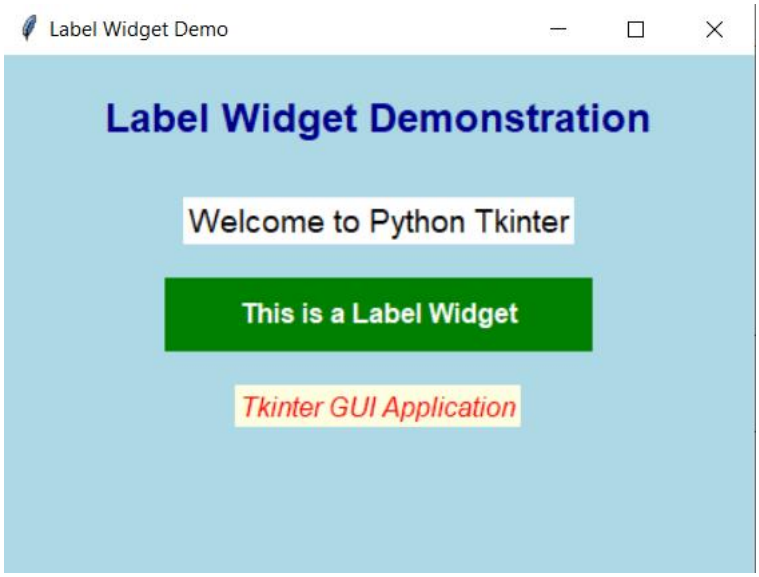
# Colored Label
label2 = Label(
   root,
   text="This is a Label Widget",
   font=("Arial", 12, "bold"),
   bg="green",
```

```
    fg="white",
    width=25,
    height=2
)
label2.pack(pady=10)

# Another Label
label3 = Label(
    root,
    text="Tkinter GUI Application",
    font=("Arial", 12, "italic"),
    bg="lightyellow",
    fg="red"
)
label3.pack(pady=10)

# Run Application
root.mainloop()
```

- **Output :-**



Q – 12 Write a code to demonstrate Listbox Widget

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Listbox Widget Demo")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Listbox Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Subject Label
label = Label(
   root,
   text="Select Your Subject:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=5)

# Create Listbox
subject_list = Listbox(
   root,
   width=25,
   height=6,
   font=("Arial", 12),
```

```
    selectmode=SINGLE,
    exportselection=False
)
subject_list.pack(pady=10)

# Insert Items into Listbox
subjects = [
    "Python",
    "Java",
    "C Programming",
    "Web Development",
    "Database",
    "Computer Network"
]

for subject in subjects:
    subject_list.insert(END, subject)

# Submit Function
def submit():
    selected = subject_list.curselection()

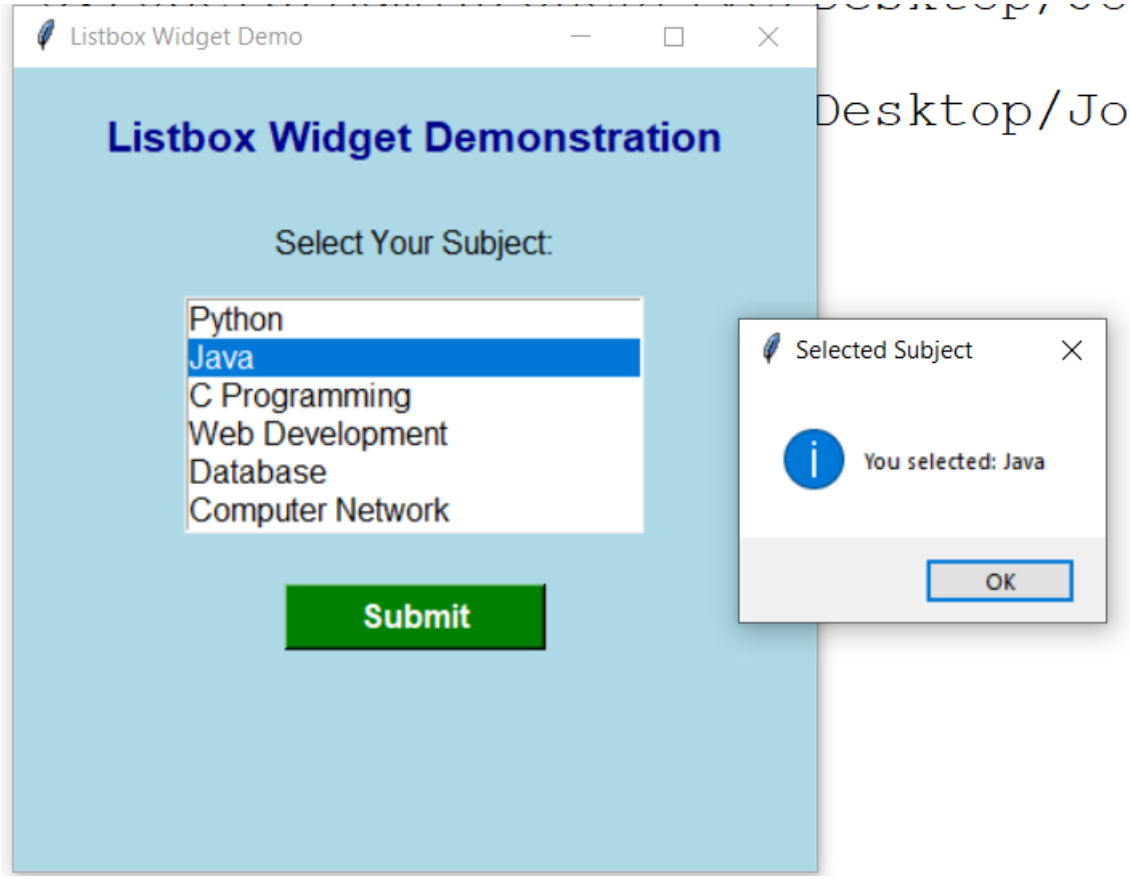
    if selected:
        subject = subject_list.get(selected[0])
        messagebox.showinfo(
            "Selected Subject",
            f"You selected: {subject}"
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please select a subject."
        )

# Submit Button
btn = Button(
    root,
    text="Submit",
```

```
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=15)

# Run Application
root.mainloop()
```

➢ **Output:-**



Q – 13 Write a code to demonstrate Radiobutton Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Radiobutton Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")

# Heading Label
heading = Label(
   root,
   text="Radiobutton Widget Demonstration",
   font=("Arial", 15, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)

# Gender Label
label = Label(
   root,
   text="Select Your Gender:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=10)

# Variable for Radiobuttons
gender_var = StringVar(value="Male")

# Male Radiobutton
rb1 = Radiobutton(
   root,
   text="Male",
   variable=gender_var,
```

```
    value="Male",
    font=("Arial", 12),
    bg="lightblue"
)
rb1.pack(anchor="w", padx=130, pady=5)

# Female Radiobutton
rb2 = Radiobutton(
    root,
    text="Female",
    variable=gender_var,
    value="Female",
    font=("Arial", 12),
    bg="lightblue"
)
rb2.pack(anchor="w", padx=130, pady=5)

# Other Radiobutton
rb3 = Radiobutton(
    root,
    text="Other",
    variable=gender_var,
    value="Other",
    font=("Arial", 12),
    bg="lightblue"
)
rb3.pack(anchor="w", padx=130, pady=5)

# Submit Function
def submit():
    gender = gender_var.get()

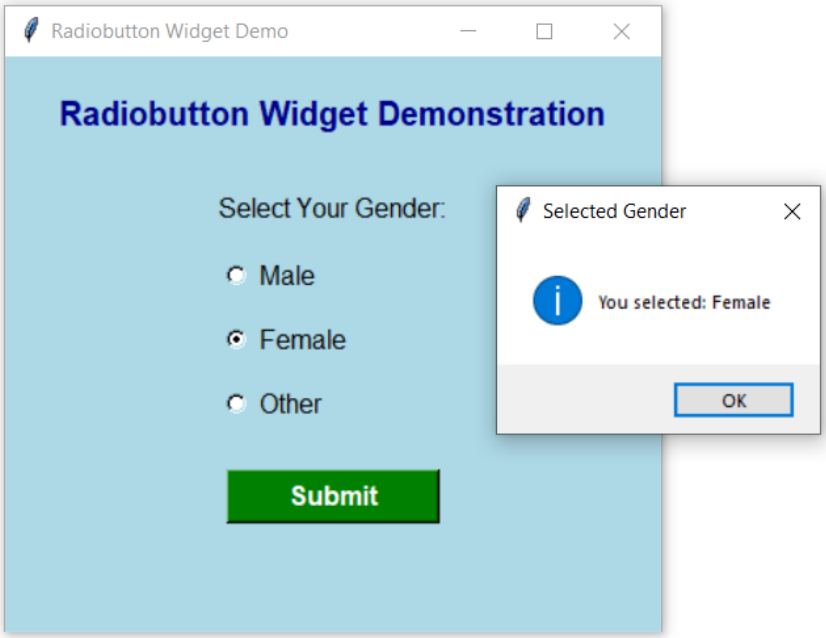
    messagebox.showinfo(
        "Selected Gender",
        f"You selected: {gender}"
    )

# Submit Button
```

```
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=20)

# Run Application
root.mainloop()
```

➢ **Output:-**



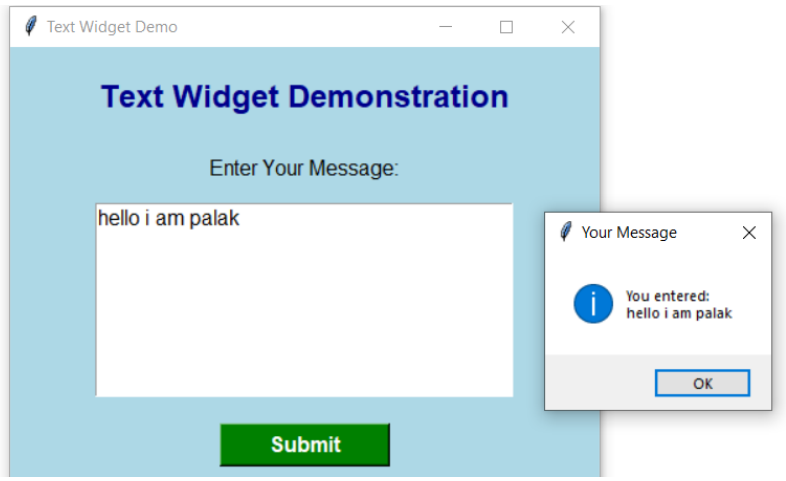
Q – 14 Write a code to demonstrate Text Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
# Main Window
root = Tk()
root.title("Text Widget Demo")
root.geometry("450x400")
root.configure(bg="lightblue")
# Heading Label
heading = Label(
   root,
   text="Text Widget Demonstration",
   font=("Arial", 18, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
label = Label(
   root,
   text="Enter Your Message:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=5)
# Create Text Widget
text_box = Text(
   root,
   width=35,
   height=8,
   font=("Arial", 12),
   wrap=WORD
)
text_box.pack(pady=10)
# Submit Function
def submit():
   message = text_box.get("1.0", END).strip()
   if message:
```

```
        messagebox.showinfo(
            "Your Message",
            f"You entered:\n{message}"
        )
    else:
        messagebox.showwarning(
            "Warning",
            "Please enter your message."
        )
# Submit Button
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=10)
# Run Application
root.mainloop()
```

➢ **Output:-**



Q – 15 Write a code to demonstrate Scale Widget in Tkinter.

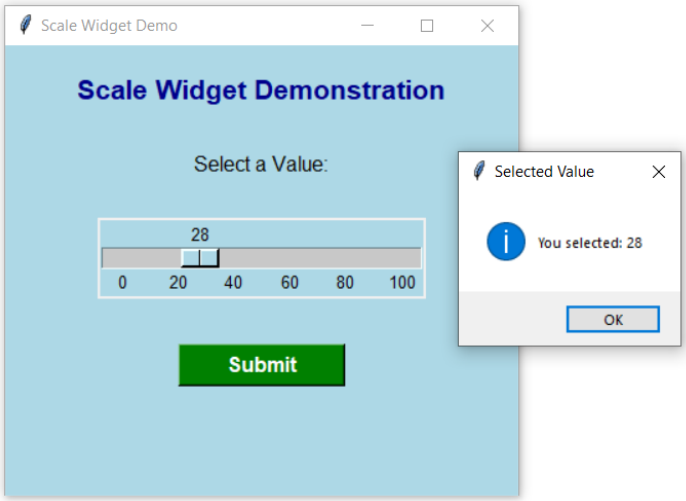
- **Input:-**

```
from tkinter import *
from tkinter import messagebox

# Main Window
root = Tk()
root.title("Scale Widget Demo")
root.geometry("400x350")
root.configure(bg="lightblue")
heading = Label(
   root,
   text="Scale Widget Demonstration",
   font=("Arial", 16, "bold"),
   bg="lightblue",
   fg="darkblue"
)
heading.pack(pady=20)
label = Label(
   root,
   text="Select a Value:",
   font=("Arial", 12),
   bg="lightblue"
)
label.pack(pady=10)
scale = Scale(
   root,
   from_=0,
   to=100,
   orient=HORIZONTAL,
   length=250,
   resolution=1,
   tickinterval=20,
   bg="lightblue",
   font=("Arial", 10)
)
scale.pack(pady=20)
# Submit Function
```

```
def submit():
    value = scale.get()
    messagebox.showinfo(
        "Selected Value",
        f"You selected: {value}"
    )
btn = Button(
    root,
    text="Submit",
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12,
    command=submit
)
btn.pack(pady=15)
# Run Application
root.mainloop()
```

- **Output:-**



Q – 16 Write a code to demonstrate Toplevel Widget in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox

root = Tk()
root.title("Main Window")
root.geometry("400x300")
root.config(bg="lightblue")

def open_window():
   new_window = Toplevel(root)
   new_window.title("New Window")
   new_window.geometry("300x200")
   new_window.config(bg="lightyellow")

   Label(
      new_window,
      text="Welcome to Toplevel Window!",
      font=("Arial", 12),
      bg="lightyellow"
   ).pack(pady=20)

   Button(
      new_window,
      text="Close Window",
      command=new_window.destroy
   ).pack(pady=10)

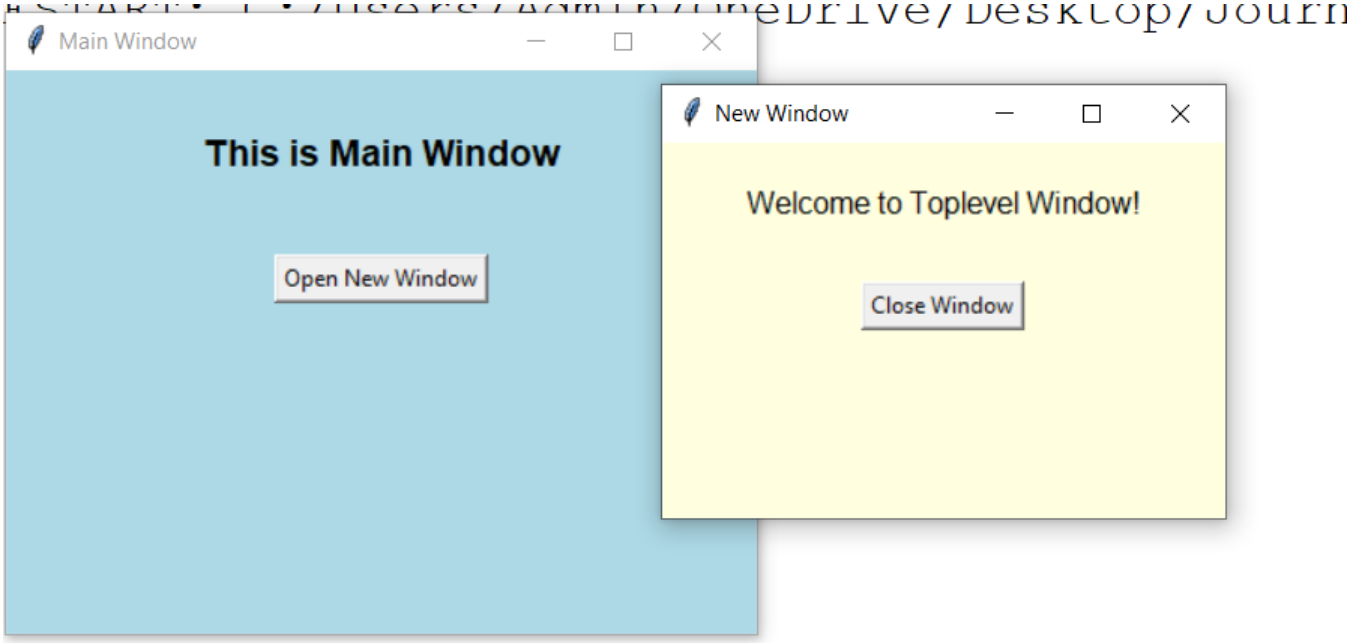
Label(
   root,
   text="This is Main Window",
   font=("Arial", 14, "bold"),
   bg="lightblue"
).pack(pady=30)

Button(
   root,
   text="Open New Window",
```

```
    command=open_window
).pack(pady=10)

root.mainloop()
```

- **Output:-**



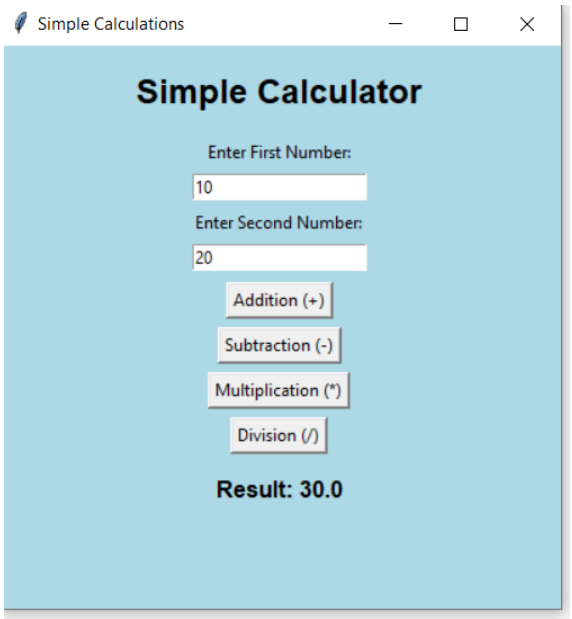
Q – 17 Write a code to demonstrate simple calculations in Tkinter.

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
root = Tk()
root.title("Simple Calculations")
root.geometry("400x400")
root.config(bg="lightblue")
def calculate(operation):
  try:
    num1 = float(entry1.get())
    num2 = float(entry2.get())
    if operation == "+":
      result = num1 + num2
    elif operation == "-":
      result = num1 - num2
    elif operation == "*":
      result = num1 * num2
    elif operation == "/":
      if num2 == 0:
        messagebox.showerror("Error", "Cannot divide by zero")
        return
      result = num1 / num2
    result_label.config(text="Result: " + str(result))
  except ValueError:
    messagebox.showerror(
      "Error", "Please enter valid numbers"
    )
Label(
  root,
  text="Simple Calculator",
  font=("Arial", 18, "bold"),
  bg="lightblue"
).pack(pady=15)
Label(root, text="Enter First Number:", bg="lightblue").pack()
entry1 = Entry(root)
entry1.pack(pady=5)
Label(root, text="Enter Second Number:", bg="lightblue").pack()
entry2 = Entry(root)
entry2.pack(pady=5)
```

```
Button(root, text="Addition (+)",
    command=lambda: calculate("+")).pack(pady=3)
Button(root, text="Subtraction (-)",
    command=lambda: calculate("-")).pack(pady=3)
Button(root, text="Multiplication (*)",
    command=lambda: calculate("*")).pack(pady=)
Button(root, text="Division (/)",
    command=lambda: calculate("/")).pack(pady=3)
result_label = Label(
  root,
  text="Result: ",
  font=("Arial", 12, "bold"),
  bg="lightblue"
)
result_label.pack(pady=10)
root.mainloop()
```

- **Output:-**



Q – 1 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Student Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("student.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS student (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   email TEXT NOT NULL,
   course TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Student Details - CRUD")
root.geometry("700x500")
root.config(bg="lightblue")
```

```
selected_id = None

# Functions
def clear_fields():
    global selected_id
    selected_id = None
    name_entry.delete(0, END)
    email_entry.delete(0, END)
    course_entry.delete(0, END)
    tree.selection_remove(tree.selection())

def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM student")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

def create_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not email or not course:
```

```
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute(
        "INSERT INTO student (name, email, course) VALUES (?, ?, ?)",
        (name, email, course)
    )
    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo("Success", "Student Added Successfully")

def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        clear_entries_only()
        name_entry.insert(0, values[1])
        email_entry.insert(0, values[2])
        course_entry.insert(0, values[3])
```

```
def clear_entries_only():
    name_entry.delete(0, END)
    email_entry.delete(0, END)
    course_entry.delete(0, END)

def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a student first"
        )
        return

    name = name_entry.get().strip()
    email = email_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not email or not course:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute(
        "UPDATE student SET name=?, email=?, course=? WHERE id=?",
        (name, email, course, selected_id)
```

```
    )
    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo("Success", "Student Updated Successfully")

def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a student first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Delete selected student?"
    )

    if answer:
        cur.execute(
            "DELETE FROM student WHERE id=?", (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo("Success", "Student Deleted Successfully")
```

```
# Heading
Label(
   root, text="Student Details Form",
   font=("Arial", 18, "bold"), bg="lightblue"
).pack(pady=10)

# Form
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Name:", bg="lightblue").grid(
   row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Email:", bg="lightblue").grid(
   row=1, column=0, padx=5, pady=5, sticky="e"
)
email_entry = Entry(form, width=30)
email_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Course:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
course_entry = Entry(form, width=30)
```

```
course_entry.grid(row=2, column=1, padx=5, pady=5)

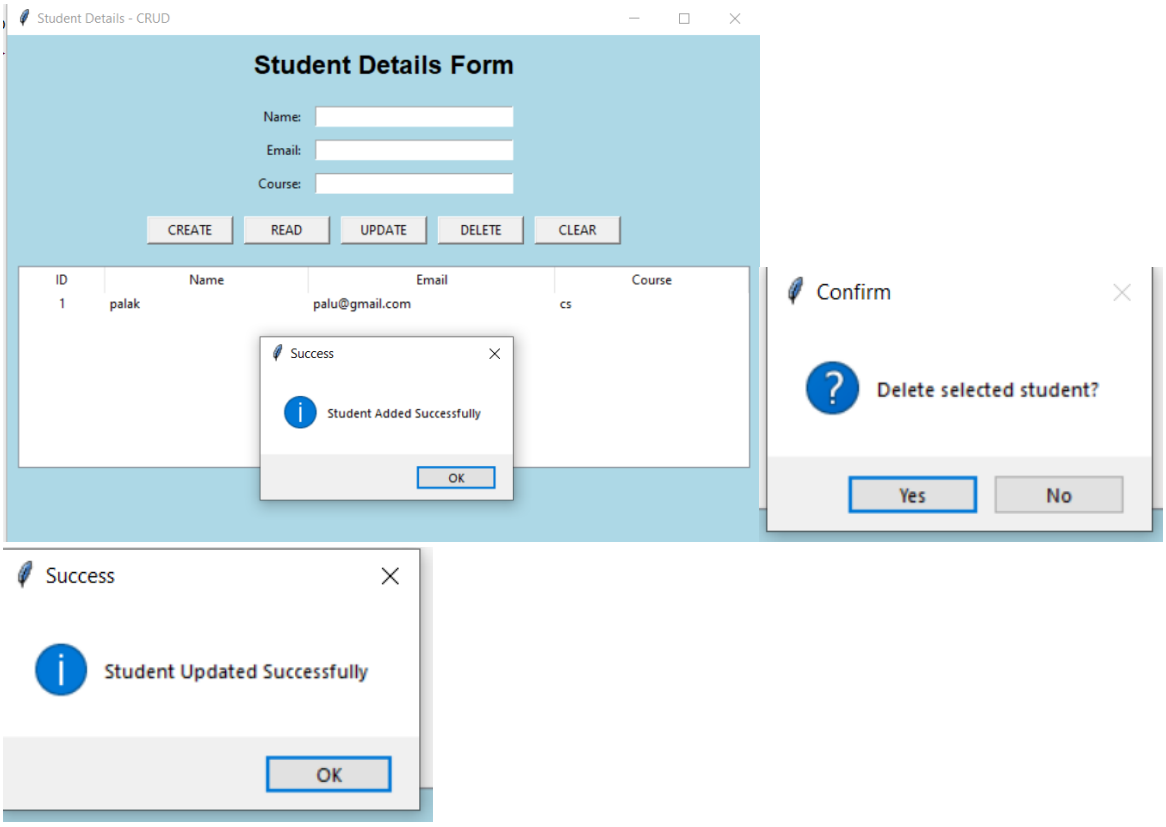

# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)


Button(buttons, text="CREATE", width=10,
    command=create_data).grid(row=0, column=0, padx=5)
Button(buttons, text="READ", width=10,
    command=read_data).grid(row=0, column=1, padx=5)
Button(buttons, text="UPDATE", width=10,
    command=update_data).grid(row=0, column=2, padx=5)
Button(buttons, text="DELETE", width=10,
    command=delete_data).grid(row=0, column=3, padx=5)
Button(buttons, text="CLEAR", width=10,
    command=clear_fields).grid(row=0, column=4, padx=5)


# Student Table
tree = ttk.Treeview(
  root,
  columns=("ID", "Name", "Email", "Course"),
  show="headings",
  height=8
)
tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
```

```
tree.heading("Email", text="Email")
tree.heading("Course", text="Course")
tree.column("ID", width=50, anchor=CENTER)
tree.column("Name", width=160)
tree.column("Email", width=200)
tree.column("Course", width=150)
tree.pack(pady=10, padx=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)
read_data()
root.mainloop()
con.close()
```

➢ Output :-



Q – 2 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Member Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("member.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS member (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   email TEXT NOT NULL,
   phone TEXT NOT NULL,
   address TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Member Details - CRUD Operations")
root.geometry("800x550")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   email_entry.delete(0, END)
   phone_entry.delete(0, END)
```

```
    address_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM member")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    phone = phone_entry.get().strip()
    address = address_entry.get().strip()

    if not name or not email or not phone or not address:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute("""
        INSERT INTO member (name, email, phone, address)
        VALUES (?, ?, ?, ?)
    """, (name, email, phone, address))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Member Added Successfully"
    )

# Select Member from Table
```

```
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        email_entry.delete(0, END)
        phone_entry.delete(0, END)
        address_entry.delete(0, END)

        name_entry.insert(0, values[1])
        email_entry.insert(0, values[2])
        phone_entry.insert(0, values[3])
        address_entry.insert(0, values[4])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a member first"
        )
        return

    name = name_entry.get().strip()
    email = email_entry.get().strip()
    phone = phone_entry.get().strip()
    address = address_entry.get().strip()

    if not name or not email or not phone or not address:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute("""
```

```
        UPDATE member
        SET name=?, email=?, phone=?, address=?
        WHERE id=?
    """, (name, email, phone, address, selected_id))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Member Updated Successfully"
    )

# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a member first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this member?"
    )

    if answer:
        cur.execute(
            "DELETE FROM member WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Member Deleted Successfully"
        )

# Heading
Label(
```

```
    root,
    text="Member Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Name:", bg="lightblue").grid(
    row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=35)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Email:", bg="lightblue").grid(
    row=1, column=0, padx=5, pady=5, sticky="e"
)
email_entry = Entry(form, width=35)
email_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
    row=2, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=35)
phone_entry.grid(row=2, column=1, padx=5, pady=5)

Label(form, text="Address:", bg="lightblue").grid(
    row=3, column=0, padx=5, pady=5, sticky="e"
)
address_entry = Entry(form, width=35)
address_entry.grid(row=3, column=1, padx=5, pady=5)

# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)
```

```
Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
).grid(row=0, column=3, padx=5)

Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

# Member Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Email", "Phone", "Address"),
   show="headings",
   height=8
)

tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
tree.heading("Email", text="Email")
tree.heading("Phone", text="Phone")
tree.heading("Address", text="Address")
```

```
tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=130)
tree.column("Email", width=180)
tree.column("Phone", width=110)
tree.column("Address", width=200)

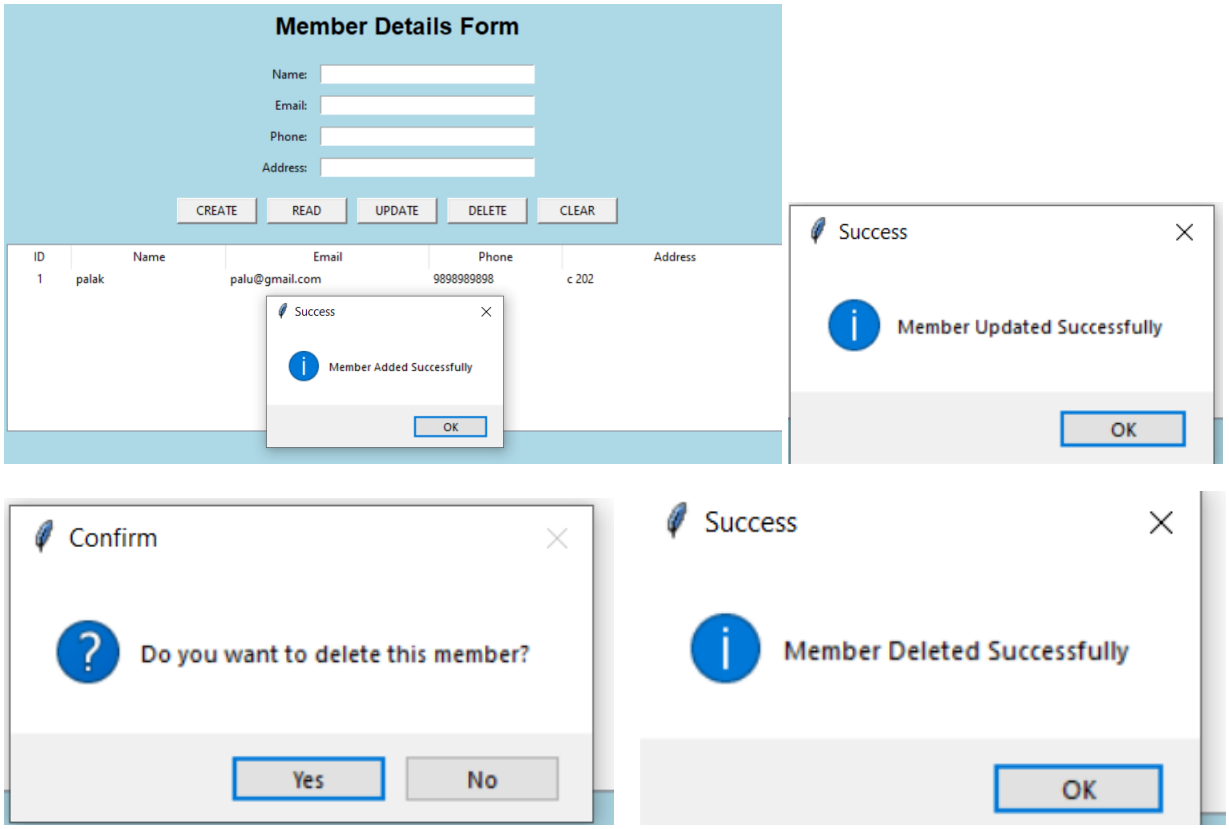
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)

# Display Existing Records
read_data()

root.mainloop()

con.close()
```

➢ **Output :-**



Q – 3 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Patient Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("patient.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS patient (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   age INTEGER NOT NULL,
   gender TEXT NOT NULL,
   disease TEXT NOT NULL,
   phone TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Patient Details - CRUD Operations")
root.geometry("850x570")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   age_entry.delete(0, END)
```

```
    gender_var.set("Male")
    disease_entry.delete(0, END)
    phone_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM patient")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gender = gender_var.get()
    disease = disease_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not age or not disease or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        age_value = int(age)
        if age_value <= 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid age"
        )
        return
```

```
    cur.execute("""
        INSERT INTO patient (name, age, gender, disease, phone)
        VALUES (?, ?, ?, ?, ?)
    """, (name, age_value, gender, disease, phone))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Patient Added Successfully"
    )

# Select Patient from Table
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        age_entry.delete(0, END)
        disease_entry.delete(0, END)
        phone_entry.delete(0, END)

        name_entry.insert(0, values[1])
        age_entry.insert(0, values[2])
        gender_var.set(values[3])
        disease_entry.insert(0, values[4])
        phone_entry.insert(0, values[5])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a patient first"
        )
```

```
    return

name = name_entry.get().strip()
age = age_entry.get().strip()
gender = gender_var.get()
disease = disease_entry.get().strip()
phone = phone_entry.get().strip()

if not name or not age or not disease or not phone:
    messagebox.showwarning(
        "Warning", "Please fill all fields"
    )
    return

try:
    age_value = int(age)
    if age_value <= 0:
        raise ValueError
except ValueError:
    messagebox.showerror(
        "Error", "Please enter a valid age"
    )
    return

cur.execute("""
    UPDATE patient
    SET name=?, age=?, gender=?, disease=?, phone=?
    WHERE id=?
""", (
    name, age_value, gender, disease, phone, selected_id
))

con.commit()
read_data()
clear_fields()
messagebox.showinfo(
    "Success", "Patient Updated Successfully"
)
```

```
# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a patient first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this patient?"
    )

    if answer:
        cur.execute(
            "DELETE FROM patient WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Patient Deleted Successfully"
        )

# Heading
Label(
    root,
    text="Patient Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Patient Name:", bg="lightblue").grid(
```

```
   row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Age:", bg="lightblue").grid(
   row=1, column=0, padx=5, pady=5, sticky="e"
)
age_entry = Entry(form, width=30)
age_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Gender:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
gender_var = StringVar(value="Male")

Radiobutton(
   form, text="Male", variable=gender_var,
   value="Male", bg="lightblue"
).grid(row=2, column=1, sticky="w")

Radiobutton(
   form, text="Female", variable=gender_var,
   value="Female", bg="lightblue"
).grid(row=2, column=1, padx=90, sticky="w")

Label(form, text="Disease:", bg="lightblue").grid(
   row=3, column=0, padx=5, pady=5, sticky="e"
)
disease_entry = Entry(form, width=30)
disease_entry.grid(row=3, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
   row=4, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=30)
phone_entry.grid(row=4, column=1, padx=5, pady=5)
```

```
# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)

Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
).grid(row=0, column=3, padx=5)

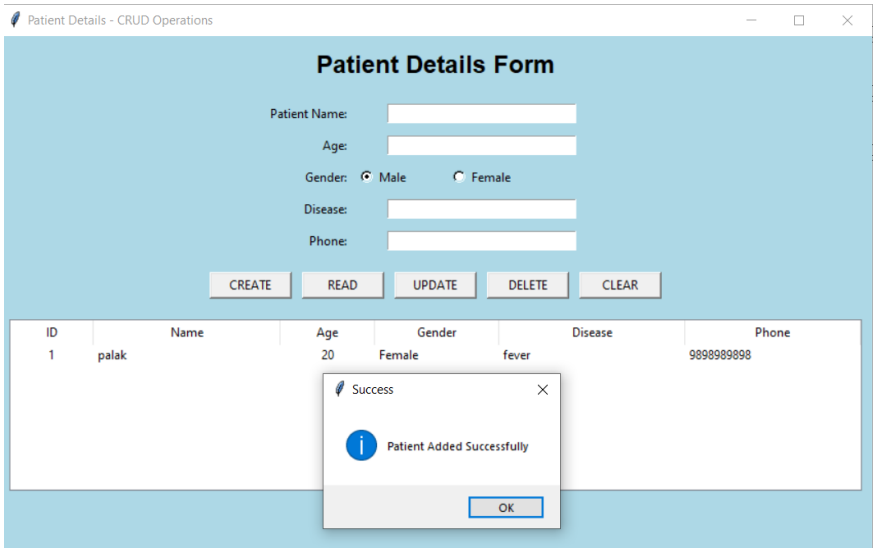
Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

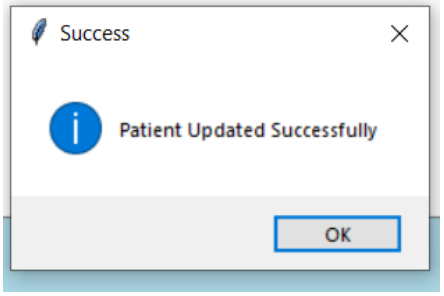
# Patient Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Age", "Gender", "Disease", "Phone"),
   show="headings",
   height=7
)

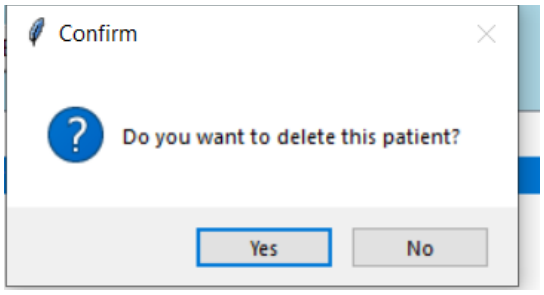
for col in ("ID", "Name", "Age", "Gender", "Disease", "Phone"):
```

```
    tree.heading(col, text=col)
tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=140)
tree.column("Age", width=50, anchor=CENTER)
tree.column("Gender", width=80)
tree.column("Disease", width=140)
tree.column("Phone", width=130)
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)
# Display Existing Records
read_data()
root.mainloop()
con.close()
```

➢ **Output :-**







Q - 4 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (emp Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("employee.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS employee (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   department TEXT NOT NULL,
   salary REAL NOT NULL,
   phone TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Employee Details - CRUD Operations")
root.geometry("850x550")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   dept_entry.delete(0, END)
   salary_entry.delete(0, END)
```

```
    phone_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM employee")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    department = dept_entry.get().strip()
    salary = salary_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not department or not salary or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        salary_value = float(salary)
        if salary_value < 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid salary"
        )
        return

    cur.execute("""
        INSERT INTO employee (name, department, salary, phone)
        VALUES (?, ?, ?, ?)
```

```
    """, (name, department, salary_value, phone))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Employee Added Successfully"
    )

# Select Employee from Table
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        dept_entry.delete(0, END)
        salary_entry.delete(0, END)
        phone_entry.delete(0, END)

        name_entry.insert(0, values[1])
        dept_entry.insert(0, values[2])
        salary_entry.insert(0, values[3])
        phone_entry.insert(0, values[4])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select an employee first"
        )
        return

    name = name_entry.get().strip()
    department = dept_entry.get().strip()
```

```
    salary = salary_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not department or not salary or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        salary_value = float(salary)
        if salary_value < 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid salary"
        )
        return

    cur.execute("""
        UPDATE employee
        SET name=?, department=?, salary=?, phone=?
        WHERE id=?
    """, (name, department, salary_value, phone, selected_id))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Employee Updated Successfully"
    )

# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select an employee first"
        )
```

```
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this employee?"
    )

    if answer:
        cur.execute(
            "DELETE FROM employee WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Employee Deleted Successfully"
        )

# Heading
Label(
    root,
    text="Employee Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Employee Name:", bg="lightblue").grid(
    row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Department:", bg="lightblue").grid(
    row=1, column=0, padx=5, pady=5, sticky="e"
```

```
)
dept_entry = Entry(form, width=30)
dept_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Salary:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
salary_entry = Entry(form, width=30)
salary_entry.grid(row=2, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
   row=3, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=30)
phone_entry.grid(row=3, column=1, padx=5, pady=5)

# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)

Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
```

```
).grid(row=0, column=3, padx=5)

Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

# Employee Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Department", "Salary", "Phone"),
   show="headings",
   height=8
)

tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
tree.heading("Department", text="Department")
tree.heading("Salary", text="Salary")
tree.heading("Phone", text="Phone")

tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=150)
tree.column("Department", width=150)
tree.column("Salary", width=120)
tree.column("Phone", width=130)

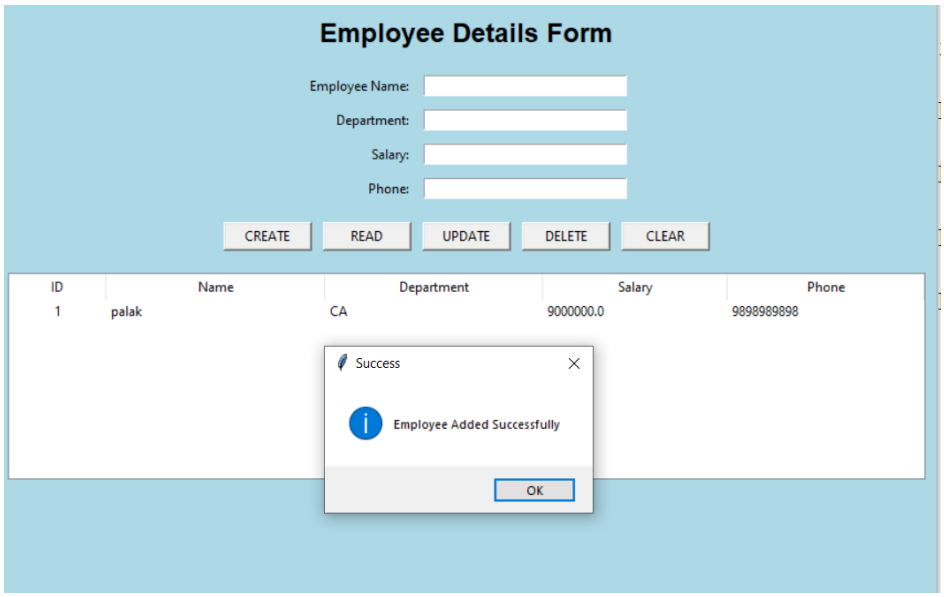
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)

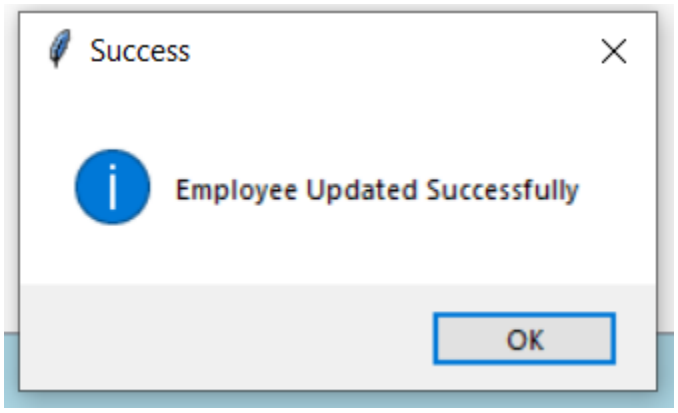
# Display Existing Records
read_data()

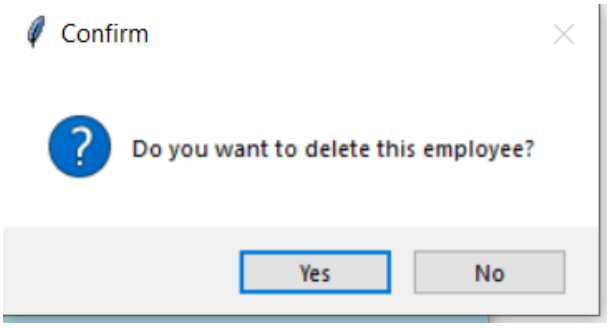
root.mainloop()

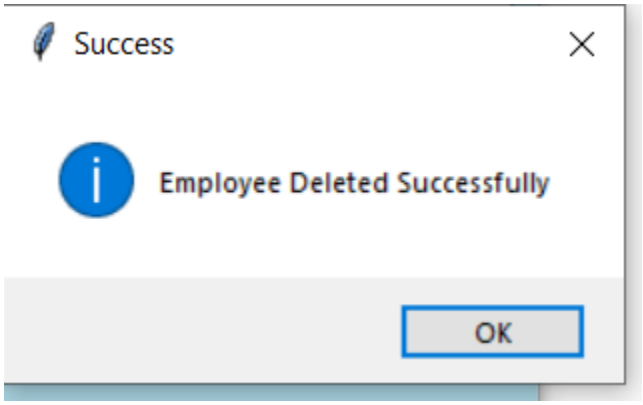
con.close()
```

- **Output :-**


Employee Details Form
Employee Name:
Department:
Salary:
Phone:
CREATE
READ
UPDATE
DELETE
CLEAR
ID
Name
Department
Salary
Phone
1
palak
CA
9000000.0
9898989898
Success
Employee Added Successfully
OK


Success
Employee Updated Successfully
OK


Confirm
Do you want to delete this employee?
Yes
No


Success
Employee Deleted Successfully
OK